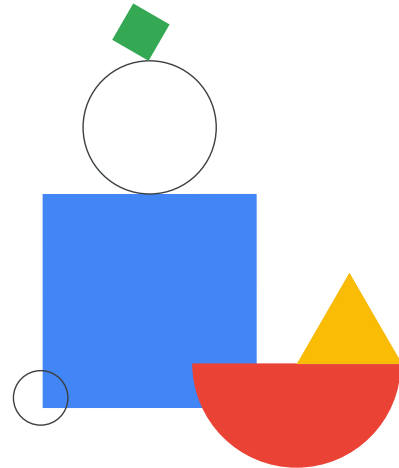


스토리지 및 데이터베이스 서비스

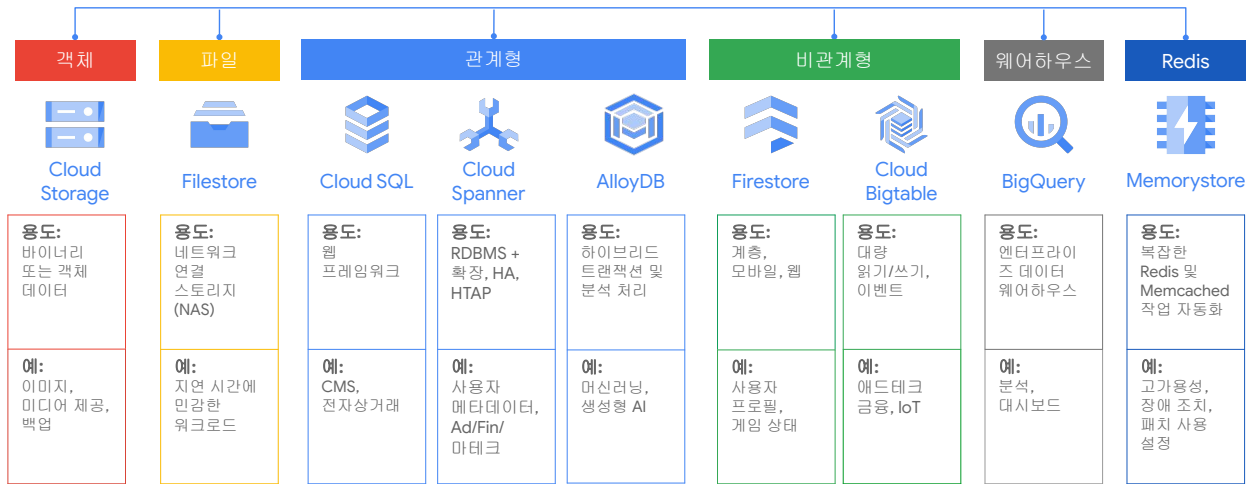


이 모듈에서는 **Google Cloud**의 스토리지와 데이터베이스 서비스에 대해 알아봅니다. 모든 애플리케이션은 비즈니스 데이터, 스트리밍할 미디어, 기기의 센서 데이터와 같은 데이터를 저장해야 합니다.

애플리케이션 중심의 관점에서 보면 이 기술은 데이터를 저장하고 검색하는 기술입니다. 데이터베이스인지 객체 저장소인지 아닌지보다 서비스의 특성을 고려했을 때 해당 서비스가 데이터의 효율적인 저장과 검색을 위한 애플리케이션의 요구사항을 지원하는지가 더 중요합니다.

Google은 여러 가지 데이터 스토리지 서비스를 제공합니다. 이 모듈에서는 **Cloud Storage**, **Filestore**, **Cloud SQL**, **Cloud Spanner**, **AlloyDB**, **Firestore**, **Cloud Bigtable**, **Memorystore**를 다룹니다. 먼저 이러한 다양한 서비스에 대해 개략적으로 설명하겠습니다.

스토리지 및 데이터베이스 서비스



이 표에서는 **Google**의 스토리지 및 데이터베이스 서비스와 스토리지 서비스 유형(객체, 파일, 관계형, 비관계형, 데이터 웨어하우스, **Redis**), 각 서비스의 용도와 사용 사례를 보여주고 있습니다.

오른쪽에는 **BigQuery**도 표시되어 있죠. 이 서비스를 언급하는 이유는 데이터 스토리지와 데이터 처리의 경계선에 위치하는 서비스이기 때문입니다. **BigQuery**에 데이터를 저장할 수 있지만 본래 **BigQuery**의 사용 목적은 빅데이터 분석 및 대화형 쿼리입니다. 따라서 **BigQuery**는 이 과정의 후반부에서 다룹니다.

범위

인프라 트랙

- 서비스 차별화 요소
- 각 서비스의 이용을 고려해야 하는 경우
- 서비스 설정 및 연결

데이터 엔지니어링 트랙

- 데이터베이스 시스템 사용 방법
- 애플리케이션의 설계, 조직, 구조, 스키마 및 용도
- 서비스가 구조화된 데이터를 저장하고 검색하는 방법에 대한 세부정보

각 데이터 스토리지 서비스에 대해 알아보기 전에 이 모듈의 범위를 정의하겠습니다.

이 모듈의 목적은 인프라의 관점에서 봤을 때 언제 어떤 서비스 사용을 고려해야 하는지 설명하는 것입니다. 데이터베이스 시스템 사용 방법을 자세히 몰라도 서비스를 설정하고 연결할 수 있을 것입니다.

설계, 조직, 구조, 스키마 및 이러한 다양한 서비스 내에서 데이터를 적절하게 최적화하고, 제공하고, 저장하는 방법을 자세히 살펴보려면 **Google Cloud**의 데이터 엔지니어링 과정을 확인해 보시기 바랍니다.

주제

01 Cloud Storage 및 Filestore

실습: Cloud Storage

02 Cloud SQL

실습: Cloud SQL 구현

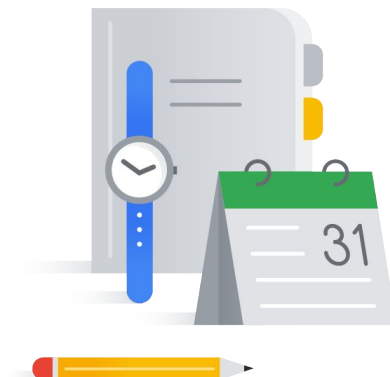
03 Cloud Spanner

04 AlloyDB

05 Firestore

06 Cloud Bigtable

07 Memorystore



Google Cloud

주제를 살펴보겠습니다. 이 모듈에서는 지금까지 언급한 모든 서비스를 다룹니다. 이러한 서비스에 더 익숙해질 수 있도록 두 번의 실습에서 적용해 보겠습니다.

또한 Google Cloud의 완전 관리형 Redis 서비스인 Memorystore에 대해서도 간략하게 설명할 것입니다.

먼저 Cloud Storage 및 Filestore를 살펴보겠습니다.



Cloud Storage 및 Filestore

객체 스토리지 서비스인 Cloud Storage

사용 사례:

- 웹사이트 콘텐츠
- 보관처리 및 재해 복구를 위한 데이터 저장
- 직접 다운로드를 통해 사용자에게 대량의 데이터 객체 배포

주요 특징:

- 엑사바이트까지 확장 가능
- 밀리초 단위의 첫 바이트 소요 시간
- 모든 스토리지 클래스에서고가용성 지원
- 모든 스토리지 클래스에서 단일 API 사용

Google Cloud

Cloud Storage는 Google Cloud의 객체 스토리지 서비스이며 데이터 양에 상관없이 언제 어디서나 데이터를 저장하고 가져올 수 있습니다. Cloud Storage의 사용 용도는 웹사이트 콘텐츠 제공, 보관처리 및 재해 복구를 위한 데이터 저장, 직접 다운로드를 통해 사용자에게 대량의 데이터 객체 배포 등 다양합니다. Cloud Storage에는 몇 가지 주요 기능이 있습니다.

- 엑사바이트 규모의 데이터로 확장 가능
- 밀리초 단위의 첫 바이트 소요 시간
- 모든 스토리지 클래스에서고가용성 지원
- 해당 스토리지 클래스 전체에 단일 API 사용

종종 Cloud Storage를 파일 시스템 내의 파일이라고 생각하기도 하지만 사실 그렇지 않습니다. Cloud Storage는 객체를 배치하는 버킷의 모음입니다. 말하자면 디렉토리를 만들 수 있지만 디렉토리는 사실 버킷의 다른 객체를 가리키는 또 다른 객체일 뿐입니다. 파일 시스템에서처럼 이러한 모든 파일을 쉽게 인덱싱할 수는 없습니다. 특정 URL만 있으면 객체에 액세스할 수 있습니다.

스토리지 클래스 개요

	Standard	Nearline	Coldline	Archive
사용 사례	데이터 집약적인 계산을 위한 '핫' 데이터 및/또는 짧은 기간만 저장되는 데이터	데이터 백업, 롱테일 멀티미디어 콘텐츠 및 데이터 보관처리와 같이 자주 액세스하지 않는 데이터	읽기 또는 수정이 분기당 최대 1회로, 자주 액세스하지 않는 데이터	데이터 보관처리, 온라인 백업 및 재해 복구
최소 저장 기간	없음	30일	90일	365일
검색 비용	없음	1GB당 \$0.01	1GB당 \$0.02	1GB당 \$0.05
가용성 SLA	99.95%(멀티/이중) 99.90%(리전)	99.90%(멀티/이중) 99.00%(리전)	99.90%(멀티/이중) 99.00%(리전)	99.90%(멀티/이중) 99.00%(리전)
내구성	99.999999999%			

Google Cloud

Cloud Storage가 제공하는 4가지 스토리지 클래스는 Standard, Nearline, Coldline 및 Archive이며, 각 스토리지 클래스는 3가지 위치 유형을 제공합니다.

- 멀티 리전은 두 개 이상의 지리적 장소를 포함하는 넓은 지리적 지역(예: 미국)입니다.
- 이중 리전은 특정한 리전의 쌍(예: 핀란드와 네덜란드)입니다.
- 리전은 특정한 지리적 장소(예: 런던)입니다.

멀티 리전 또는 이중 리전에 저장된 객체는 지리적 중복입니다. 이제 각 스토리지 클래스를 살펴보겠습니다.

- **Standard Storage**는 액세스 빈도가 높거나('핫' 데이터) 짧은 기간만 저장되는 데이터에 가장 적합합니다. 가장 비싼 스토리지 클래스이지만 저장 기간이 가장 짧고 검색 비용이 없습니다. 사용 사례는 다음과 같습니다.
 - 리전에서 사용하는 경우, **Standard Storage**는 데이터를 사용하는 **Google Kubernetes Engine** 클러스터 또는 **Compute Engine** 인스턴스와 동일한 위치에 데이터를 저장하기에 적합합니다. 리소스를 공동으로 배치하면 데이터 집약적인 계산의 성능을 극대화하고 네트워크 비용을 줄일 수 있습니다.
 - 이중 리전에서 사용하는 경우, 연결된 리전 중 하나에 위치한 **Google Cloud** 제품에 액세스할 때도 성능이 최적화되지만, 지리적으로 별도의 위치에 데이터를 저장하는 경우에도 가용성이 향상됩니다.

- 멀티 리전에서 사용하는 경우, **Standard Storage**는 웹사이트 콘텐츠 제공, 스트리밍 동영상, 대화형 워크로드 실행, 모바일 및 게임 애플리케이션을 지원하는 데이터 제공과 같이 전 세계에서 자주 액세스되는 데이터를 저장하는 데 적합합니다.
- **Nearline Storage**는 데이터 백업, 롱테일 멀티미디어 콘텐츠, 데이터 보관처리와 같이 액세스 빈도가 낮은 데이터를 저장하는 데 사용하는 저렴한 비용에 내구성 높은 스토리지 서비스입니다. 데이터 스토리지 비용을 줄이기 위해 조금은 낮은 가용성, 30일의 최소 저장 기간, 데이터 액세스 비용을 용인할 수 있다면 **Nearline Storage**가 **Standard Storage**보다 더 나은 선택입니다.
- **Coldline Storage**는 자주 액세스하지 않는 데이터를 저장하는 데 사용하는 매우 저렴하고 내구성 높은 스토리지 서비스입니다. 데이터 스토리지 비용을 줄이기 위해 조금은 낮은 가용성, 90일의 최소 저장 기간, 높은 데이터 액세스 비용을 용인할 수 있다면 **Coldline Storage**가 **Standard Storage**나 **Nearline Storage**보다 더 나은 선택입니다.
- **Archive Storage**는 데이터 보관처리, 온라인 백업, 재해 복구에 사용하는 가장 저렴하고 내구성 높은 스토리지 서비스입니다. 다른 클라우드 공급업체에서 제공하는 가장 높은 수준의 '콜드' 스토리지 서비스와 달리 데이터는 몇 시간 또는 며칠이 아니라 밀리초 이내에 제공됩니다. **Archive Storage**는 또한 데이터 액세스 및 운영 비용이 상대적으로 높은 편이며, 최소 저장 기간도 365일입니다. **Archive Storage**는 1년에 한 번 미만으로 액세스하려는 데이터에 적합합니다.

내구성과 가용성에 초점을 맞춰보겠습니다. 이 모든 스토리지 클래스는 99.999999999%의 내구성을 제공합니다. 이것이 무슨 뜻일까요? 언제나 파일에 액세스할 수 있다는 뜻일까요? 아닙니다. 이것은 데이터가 손실되지 않는다는 의미입니다. 데이터에 액세스하지 못할 수도 있습니다. 은행에 돈을 맡겨 두고 "내 돈은 은행에 있어"라고 말하는 것과 같죠. 이것이 99.999999999% 내구성입니다. 하지만 은행이 문을 닫으면 돈을 찾을 수 없겠죠? 이것이 바로 가용성입니다. 가용성은 스토리지 클래스와 위치 유형에 따라 다릅니다.

Cloud Storage 개요

버킷

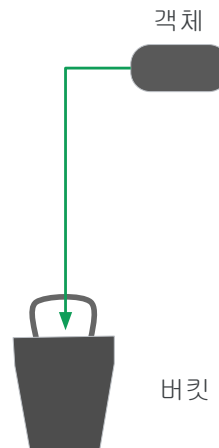
- 이름 지정 요구사항
- 중첩될 수 없음

객체

- 생성 시 버킷의 스토리지 클래스 상속
- 최소 크기 없음, 무제한 저장용량

액세스

- `gcloud storage` 명령어
- (RESTful) JSON API 또는 XML API



Cloud Storage는 몇 가지 항목으로 나눌 수 있습니다.

- 우선, 버킷이 있습니다. 버킷은 전역적으로 고유한 이름이 필요하고, 중첩될 수 없습니다.
- 버킷에 들어가는 데이터는 버킷의 스토리지 클래스를 상속하는 객체이며, 이러한 객체는 텍스트 파일, 문서 파일, 비디오 파일 등이 될 수 있습니다. 이러한 객체에는 최소 크기가 없으며 할당량이 허용하는 한 원하는 만큼 확장할 수 있습니다.
- `gcloud storage` 명령어, 또는 JSON이나 XML API를 사용하여 데이터에 액세스할 수 있습니다.

기본 스토리지 클래스의 변경

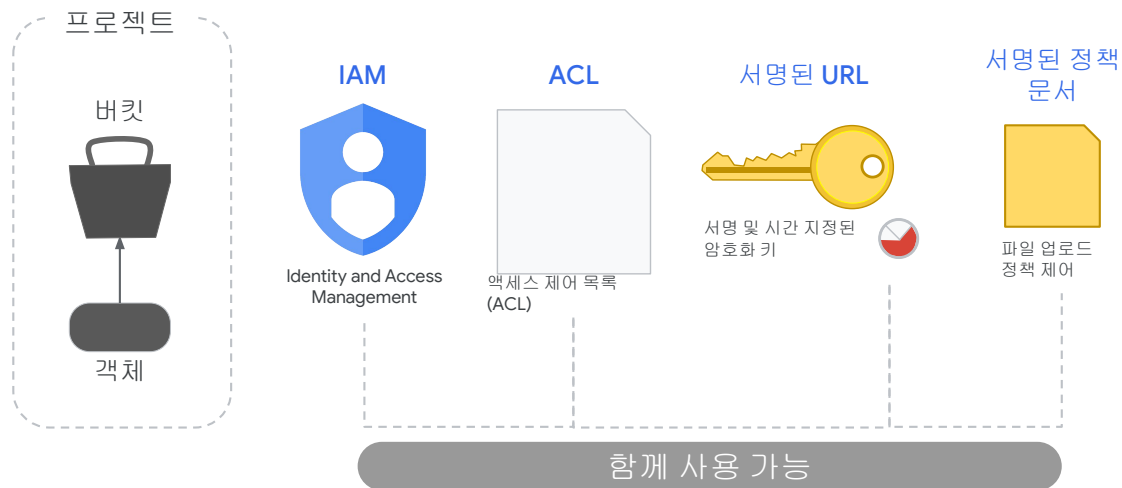
- 새 객체에는 기본 클래스가 적용됩니다.
- 리전 버킷은 멀티 리전/이중 리전으로 변경할 수 없습니다.
- 멀티 리전 버킷은 리전으로 변경할 수 없습니다.
- 버킷에서 버킷으로 객체를 이동할 수 있습니다.
- 객체 수명 주기 관리 기능으로 객체 클래스를 관리할 수 있습니다.

버킷에 객체를 업로드할 때 객체의 스토리지 클래스를 지정하지 않으면 버킷의 스토리지 클래스가 객체에 할당됩니다. 버킷의 기본 스토리지 클래스를 변경할 수 있지만 위치 유형을 리전에서 멀티 리전/이중 리전으로 변경하거나 그 반대로 변경할 수는 없습니다.

객체를 다른 버킷으로 이동하거나 객체의 URL을 변경하지 않은 채, 이미 버킷에 있는 객체의 스토리지 클래스를 변경할 수도 있습니다. 자주 액세스하지 않을 것으로 예상되는 객체를 버킷에 보관하려는 경우 객체별로 스토리지 클래스를 설정하면 유용합니다. 이 경우 해당 객체의 스토리지 클래스를 **Nearline**, **Coldline** 또는 **Archive Storage**로 변경하여 비용을 최소화할 수 있습니다.

버킷의 객체 클래스를 관리할 수 있도록 **Cloud Storage**는 객체 수명 주기 관리 기능을 제공합니다. 이에 대해서는 나중에 자세히 설명하겠습니다.

액세스 제어



Google Cloud

프로젝트의 일부인 객체 및 버킷에 대한 액세스 제어를 살펴보겠습니다.

- 프로젝트에 **IAM**을 사용하여 개별 사용자 또는 서비스 계정이 버킷을 보거나, 버킷의 객체를 나열하거나, 버킷에 있는 객체의 이름을 보거나, 새 버킷을 생성할 수 있는 권한을 제어할 수 있습니다. 대부분의 경우 **IAM**으로 충분하며 역할은 프로젝트에서 버킷, 객체로 상속됩니다.
- 액세스 제어 목록(**ACL**)은 더 세밀한 제어를 제공합니다.
- 더 세부적으로 제어가 가능한, 서명된 **URL**은 암호화 키를 제공합니다. 암호화 키는 버킷 또는 객체에 대한 액세스 시간을 제한하죠.
- 마지막으로 서명된 정책 문서는 서명된 **URL**이 있는 사람이 업로드할 수 있는 파일의 종류를 정하는 방식으로 더욱더 정교한 제어가 가능합니다. **ACL**과 서명된 **URL**을 자세히 살펴보겠습니다.

액세스 제어 목록(ACL)



Google Cloud

ACL은 버킷 및 객체에 액세스할 수 있는 사용자와 해당 사용자에게 제공되는 액세스 권한 수준을 정의하는 데 사용할 수 있는 메커니즘입니다. 버킷이나 객체마다 최대 100개의 ACL 항목을 만들 수 있습니다.

각 ACL은 하나 이상의 항목으로 이루어지며 이러한 항목은 다음 두 가지 정보로 구성됩니다.

- 범위: 지정된 작업을 수행할 수 있는 사람을 정의합니다(예: 특정 사용자 또는 사용자 그룹).
- 권한: 수행할 수 있는 작업을 정의합니다(예: 읽기 또는 쓰기).

이 슬라이드에 나와 있는 **allUsers** 식별자는 Google 계정의 유무와 관계없이 인터넷에 있는 모든 사람을 나타냅니다. 이와 대조적으로 **allAuthenticatedUsers** 식별자는 Google 계정으로 인증된 모든 사람을 나타냅니다.

ACL에 대한 자세한 내용은 [문서](#)를 참고하세요.

서명된 URL

- 티켓을 통해 이루어지는 버킷 및 객체에 대한 '발렛 키' 액세스:
 - 암호화 방식으로 서명된 URL인 티켓
 - 시간 제한
 - 티켓에 지정된 작업: HTTP GET, PUT, DELETE(POST는 아님)
 - URL이 있는 모든 사용자는 허용된 작업을 호출 가능
- 개인 계정 키 및 `gcloud storage` 사용 예시:
`gcloud storage signurl -d 10m path/to/privatekey.p12`
`gs://bucket/object`

Google Cloud

일부 애플리케이션의 경우, 리소스 액세스를 제어하는 데 계정 기반 인증을 사용하는 대신 모든 사용자가 사용 가능한 시간이 제한된 액세스 토큰을 부여하는 것이 더 쉽고 효율적입니다. (예를 들어 사용자에게 Google 계정을 요구하지 않으려고 하는 경우)

서명된 URL을 사용하면 Cloud Storage에 대해 이 작업을 수행할 수 있습니다. 특정 Cloud Storage 리소스에 대한 읽기 또는 쓰기 액세스 권한을 부여하고 액세스 만료 시기를 지정하는 URL을 생성합니다. 해당 URL은 서비스 계정과 연결된 비공개 키를 사용하여 서명됩니다. 요청이 수신되면 Cloud Storage는 액세스 권한 부여 URL이 신뢰할 수 있는 보안 주체(이 경우 서비스 계정)를 대신하여 발급되었는지 확인하고 해당 계정의 신뢰를 URL 소유자에게 위임할 수 있습니다.

서명된 URL을 제공한 후에는 제어할 수 없습니다. 따라서 적절한 시간이 지나면 서명된 URL이 만료되도록 하는 것이 좋습니다.

Cloud Storage의 기능

- 고객 제공 암호화 키(CSEK)
 - Google이 관리하는 키 대신 자체 키 사용
- 객체 수명 주기 관리
 - 자동으로 객체 삭제 또는 보관처리
- 객체 버전 관리
 - 여러 버전의 객체 유지
- 디렉터리 동기화
 - VM 디렉터리를 버킷과 동기화
- Pub/Sub를 사용한 객체 변경 알림
- 자동 클래스

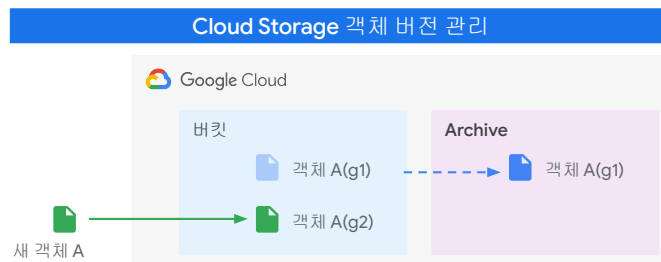
Google Cloud

Cloud Storage와 함께 제공되는 몇 가지 기능도 있습니다. 조만간 이 기능 중 일부에 대해 더 자세히 알아볼 것이기 때문에 지금은 개략적으로 소개하겠습니다.

- 이 과정의 앞부분에서 가상 머신에 영구 디스크를 연결할 때 고객 제공 암호화 키에 대해 조금 이야기했습니다. 이를 통해 Google이 관리하는 키 대신 자체 암호화 키를 제공할 수 있으며 Cloud Storage에서도 사용할 수 있습니다.
- Cloud Storage는 또한 객체를 자동으로 삭제하거나 보관처리할 수 있는 객체 수명 주기 관리를 제공합니다.
- 또 다른 기능은 버킷에 있는 여러 버전의 객체를 유지할 수 있는 객체 버전 관리입니다. 여러 개의 파일처럼 여러 버전에 대해 요금이 부과된다는 점을 명심하세요.
- Cloud Storage는 디렉터리 동기화도 제공하므로 VM 디렉터리를 버킷과 동기화할 수 있습니다.
- Pub/Sub를 사용하여 Cloud Storage에 대한 객체 변경 알림을 구성할 수 있습니다.
- 사용 설정하면 자동 클래스가 버킷의 스토리지 클래스를 모든 측면에서 관리합니다. 이 부분에 대해서는 나중에 살펴보겠습니다.

삭제되거나 덮어쓰는 객체 검색을 지원하는 객체 버전 관리

- 객체는 변경 불가
- 객체 버전 관리:
 - 객체 수정 이력을 기록합니다.
 - 객체의 보관처리된 버전을 나열하거나 객체를 이전 상태로 복원하거나 버전을 삭제합니다.



Google Cloud

Cloud Storage에서는 객체를 변경하는 것이 불가능합니다. 즉, 스토리지 수명 주기 동안 업로드한 객체를 변경할 수 없죠. 삭제하거나 덮어쓰는 객체를 검색할 수 있도록 Cloud Storage는 객체 버전 관리 기능을 제공합니다. 버킷에 대해 객체 버전 관리 기능을 사용 설정할 수 있습니다. 사용 설정되면 객체의 실시간 버전을 덮어쓰거나 삭제할 때마다 Cloud Storage가 보관처리된 객체 버전을 만듭니다. 보관처리된 버전의 객체 이름은 유지되지만 이 슬라이드에서 보이는 g1처럼 세대 번호로 고유하게 식별됩니다.

객체 버전 관리를 사용 설정하면 객체의 보관처리된 버전을 나열하거나, 객체의 실시간 버전을 이전 상태로 복원하거나, 필요에 따라 보관처리된 버전을 영구 삭제할 수 있습니다. 언제든지 버킷의 버전 관리를 설정하거나 해제할 수 있습니다. 버전 관리를 중지하면 기존 객체 버전은 남지만 버킷이 보관처리된 새로운 객체 버전을 더 이상 추적하지 않습니다.

객체 버전 관리에 대한 자세한 내용은 [문서](#)를 참고하세요.

특정 규칙을 충족하는 객체에 대해 수행할 작업을 지정하는 객체 수명 주기 관리 정책

- 예:
 - 1년 이상 된 객체의 스토리지 클래스를 다운그레이드합니다.
 - 특정 날짜 이전에 생성된 객체를 삭제합니다.
 - 객체에 대해 가장 최근의 3개 버전만 유지합니다.
- 객체 검사는 비동기식 배치에서 발생합니다.
- 변경사항을 적용하려면 최대 24시간이 걸릴 수 있습니다.

Google Cloud

Cloud Storage는 객체 수명 주기 관리를 통해 객체의 **TTL(수명)** 설정, 이전 버전의 객체 보관처리, 비용 관리를 위한 객체 스토리지 클래스 '다운그레이드'와 같은 일반 사용 사례를 지원합니다.

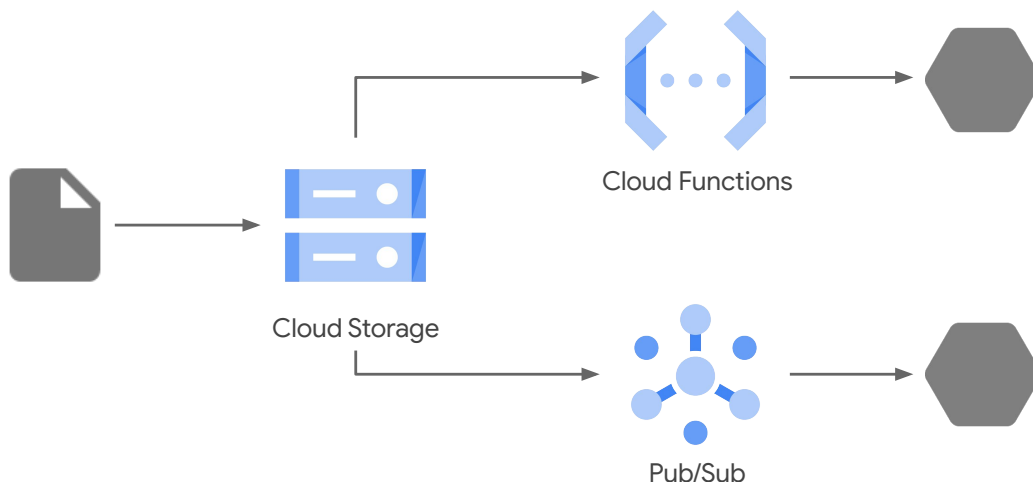
수명 주기 관리 구성을 버킷에 할당할 수 있습니다. 구성은 버킷의 모든 객체에 적용되는 규칙 집합입니다. 따라서 객체가 한 규칙의 조건에 맞으면 **Cloud Storage**가 객체에서 지정된 작업을 자동으로 수행합니다. 몇 가지 사용 사례는 다음과 같습니다.

- 1년 이상 된 객체의 스토리지 클래스를 **Archive Storage**로 다운그레이드합니다.
- 특정 날짜 이전에 생성된 객체를 삭제합니다. (예: 2017년 1월 1일)
- 버전 관리가 사용 설정된 상태에서는 버킷 내 각 객체에 대해 가장 최근의 3개 버전만 유지합니다.

객체 검사는 비동기식 배치에서 발생하므로 규칙은 즉시 적용되지 않을 수 있습니다. 또한 수명 주기 구성의 업데이트가 반영되기까지 최대 24시간이 걸릴 수 있습니다. 즉, 수명 주기 구성을 변경할 때 객체 수명 주기 관리가 최대 24시간 동안 여전히 이전 구성을 기준으로 작업을 수행할 수도 있다는 의미입니다. 이 점에 유의하시기 바랍니다.

객체 수명 주기 관리에 대한 자세한 내용은 [문서](#)를 참고하세요.

Cloud Storage용 Cloud Pub/Sub 알림



Google Cloud

Pub/Sub를 사용하여 **Cloud Storage**에 대한 객체 변경 알림을 구성할 수 있습니다. **Pub/Sub** 알림은 버킷의 객체 변경사항에 대한 정보를 **Pub/Sub**로 전송하며, 여기에서 정보는 메시지의 형태로 선택한 **Pub/Sub** 주제에 추가됩니다. 예를 들어, 버킷에서 만들어지고 삭제되는 객체를 추적할 수 있습니다. 각 알림에는 알림을 트리거한 이벤트와 변경된 객체를 설명하는 정보가 포함됩니다. 프로젝트에 대한 권한이 충분한 경우 어떤 **Pub/Sub** 주제에도 알림을 전송할 수 있습니다. **Pub/Sub** 주제로 수신된 후에는 결과 메시지를 모든 주제 구독자에게 전송할 수 있습니다. **Cloud Storage** 버킷을 **Pub/Sub** 주제에 연결하는 방법에 대한 정보는 [기본 요건](#) 문서를 참조하세요.

Pub/Sub 알림은 더 빠르고 유연하며 설정하기 쉽고 경제적이므로 **Cloud Storage** 버킷 내 객체 변경사항을 추적하는 데 권장되는 방법입니다. **Pub/Sub**는 Google의 분산형 실시간 메시징 서비스로, 애플리케이션 개발에 대한 교육 과정에서 다룹니다.

데이터 가져오기 서비스

- **Transfer Appliance:** 데이터를 랙 처리하고, 캡처한 후 Google Cloud로 전달합니다.
- **Storage Transfer Service:** 온라인 데이터(다른 버킷, S3 버킷 또는 웹 소스)를 가져옵니다.
- 오프라인 미디어 가져오기: 타사 제공업체가 물리적 매체에서 데이터를 업로드합니다.



Google Cloud

Google Cloud 콘솔을 사용하여 버킷에 개별 파일을 업로드할 수 있습니다. 테라바이트 또는 페타바이트의 데이터를 업로드해야 하는 경우에는 어떻게 해야 할까요? 이 문제를 해결하는 세 가지 서비스가 있습니다. 바로 **Transfer Appliance**, **Storage Transfer Service**, 오프라인 미디어 가져오기입니다.

- [Transfer Appliance](#)는 비즈니스 운영을 중단하지 않고도 수백 테라바이트에서 최대 1페타바이트에 이르는 대량의 데이터를 Google Cloud로 안전하게 마이그레이션할 때 사용할 수 있는 하드웨어 어플라이언스입니다. 이 슬라이드에 나와 있는 이미지가 **Transfer Appliance**의 이미지입니다.
- [Storage Transfer Service](#)를 사용하면 온라인 데이터를 고성능으로 가져올 수 있습니다. 데이터를 가져오게 될 소스는 다른 **Cloud Storage** 버킷, **Amazon S3** 버킷 또는 **HTTP/HTTPS** 위치일 수 있습니다.
- 마지막으로 [오프라인 미디어 가져오기](#)는 데이터를 업로드하는 제공업체에게 물리적 매체(예: 스토리지 배열, 하드 디스크 드라이브(HDD), 테이프 및 USB 플래시 드라이브)를 전달하는 타사 서비스입니다.

전역적 **strong consistency**를 제공하는 Cloud Storage

- 쓰기 후 읽기
- 메타데이터 업데이트 후 읽기
- 삭제 후 읽기
- 버킷 나열
- 객체 나열



Google Cloud

Cloud Storage에 객체를 업로드하고 성공 응답을 받으면 Google이 서비스를 제공하는 모든 위치에서 객체를 즉시 다운로드하고 메타데이터 작업에 사용할 수 있습니다. 새로운 객체를 만들거나 기존 객체를 덮어쓸 때도 마찬가지입니다. 업로드는 **strong consistency**를 가지므로, 쓰기 후 읽기 또는 메타데이터 업데이트 후 읽기 작업에 대해 **404 Not Found** 응답이나 비활성 데이터를 수신하는 일이 절대 없습니다.

전역적인 **strong consistency**는 객체에 대한 삭제 작업으로까지도 확장됩니다. 삭제 요청이 성공한 경우, 객체 또는 객체의 메타데이터를 즉시 다운로드하려고 시도하면 **404 Not Found** 상태 코드가 반환됩니다. 삭제 작업이 성공한 후에는 객체가 더 이상 존재하지 않으므로 **404** 오류가 발생합니다.

버킷 나열은 **strong consistency**를 가집니다. 예를 들어, 버킷을 만든 후 버킷 나열 작업을 즉시 수행하면 반환된 버킷 목록에 새로운 버킷이 나타납니다.

객체 나열도 **strong consistency**를 가집니다. 마찬가지로, 객체를 버킷에 업로드한 후 객체 나열 작업을 즉시 수행하면 반환된 객체 목록에 새로운 객체가 나타납니다.

스토리지 클래스 선택하기



Google Cloud

Cloud Storage에서 적절한 스토리지 클래스를 찾는 데 도움이 되는 결정 트리를 살펴보겠습니다.

- 데이터를 1년에 한 번 미만으로 읽는 경우 **Archive Storage** 사용을 고려하는 것이 좋습니다.
- 데이터를 90일에 한 번 미만으로 읽는 경우 **Coldline Storage** 사용을 고려하는 것이 좋습니다.
- 데이터를 30일에 한 번 미만으로 읽는 경우 **Nearline Storage** 사용을 고려하는 것이 좋습니다.
- 그리고 이보다 읽기, 쓰기를 더 자주 수행할 경우 **Standard Storage** 사용을 고려하는 것이 좋습니다.

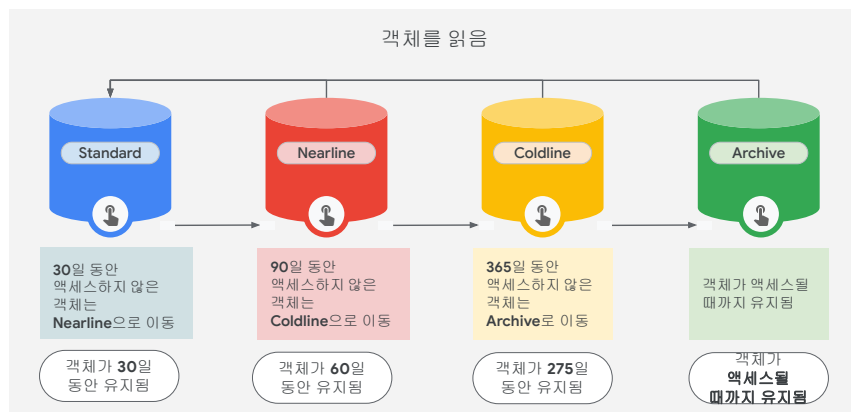
위치 유형도 고려해야 합니다.

- 분석 파이프라인과 같이 동일한 리전에서 그룹화된 데이터 소비자의 지연 시간 및 네트워크 대역폭을 최적화하려면 **리전**을 사용합니다.
- 리전과 유사한 성능 이점과 함께 지리적 중복을 통해 제공되는 더 높은 가용성을 원하는 경우 **이중 리전**을 사용합니다.
- Google 네트워크 외부에 있거나 넓은 지리적 지역에 분산되어 있는 데이터 소비자에게 콘텐츠를 제공하려는 경우 또는 지리적 중복을 통해 제공되는 더 높은 가용성을 원하는 경우에는 **멀티 리전**을 사용합니다.

데이터의 액세스 빈도가 다양하거나, 데이터의 액세스 패턴을 알 수 없거나, 예측할 수 없는 경우 자동 클래스를 고려해야 합니다.

Google Cloud의 자동 클래스 스토리지

자동 클래스는 버킷의 객체를 각 객체의 액세스 패턴을 기준으로 적절한 스토리지 클래스로 전환합니다.



Google Cloud

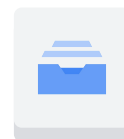
자동 클래스 기능은 버킷의 객체를 각 객체의 액세스 패턴을 기준으로 적절한 스토리지 클래스로 자동 전환합니다. 요청에 다른 스토리지 클래스가 지정되어 있어도 버킷에 추가된 모든 객체는 **Standard Storage**에서 시작됩니다. 이 기능은 스토리지 비용을 줄일 수 있도록 액세스하지 않는 데이터를 사용량이 적은 스토리지 클래스로 이동합니다. 액세스하는 데이터는 향후 액세스 최적화를 위해 **Standard Storage**로 이동됩니다.

객체 데이터를 읽을 때 객체가 아직 **Standard Storage**에 저장되지 않은 경우 **Standard Storage**로 전환됩니다. 자동 클래스를 사용하면 **Cloud Storage** 데이터 비용을 간단하게 자동으로 절감할 수 있습니다. 버킷에서 사용 설정하면 조기 삭제 요금, 검색 요금, 스토리지 클래스 전환 요금이 청구되지 않습니다.

자세한 내용은 [스토리지 클래스](#)를 참조하세요.

지금까지 우리는 구조화되지 않은 데이터만 고려했습니다. 구조화되지 않은 데이터를 살펴보기 전에 고성능의 완전 관리형 파일 서비스인 **Filestore**를 살펴보겠습니다.

애플리케이션을 위한 관리형 파일 저장 서비스인 **Filestore**



Filestore

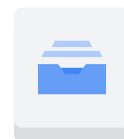
- Compute Engine 및 GKE 인스턴스용 완전 관리형 네트워크 연결 스토리지(NAS)
- 예측 가능한 성능
- 완전한 NFSv3 지원
- 고성능 워크로드를 위해 수백 TB로 확장

Google Cloud

Filestore는 파일 시스템 인터페이스와 데이터용 공유 파일 시스템이 필요한 애플리케이션을 위해 만들어진 관리형 파일 저장 서비스입니다. **Filestore**는 관리형 네트워크 연결 스토리지 (NAS)를 **Compute Engine** 및 **Google Kubernetes Engine** 인스턴스와 같이 구축하는 데 편리한 기본 환경을 제공합니다. **Filestore**의 성능과 용량을 개별적으로 미세하게 조정할 수 있어 파일 기반 워크로드에서 빠른 성능을 얻을 수 있습니다.

Filestore는 기본적으로 기존 엔터프라이즈 애플리케이션과 호환되며 **NFSv3** 호환 클라이언트를 지원합니다. 전문적인 플러그인 또는 클라이언트 측 소프트웨어를 설치하거나 유지관리할 필요 없이 애플리케이션에서 수평 확장 성능, 수백 TB의 용량, 파일 잠금 등의 기능을 활용할 수 있습니다.

Filestore의 다양한 사용 사례



Filestore

- 애플리케이션 마이그레이션
- 미디어 렌더링
- 전자 설계 자동화(EDA)
- 데이터 분석
- Genomics 처리
- 웹 콘텐츠 관리

Filestore의 사용 사례는 다양합니다. **Filestore**를 사용하여 엔터프라이즈 애플리케이션의 마이그레이션을 신속하게 처리할 수 있습니다. 다양한 온프레미스 애플리케이션에는 데이터를 관리할 수 있는 파일 시스템 인터페이스가 필요합니다. 이러한 애플리케이션이 계속해서 클라우드로 마이그레이션됨에 따라 **Filestore**는 공유 파일 시스템이 필요한 엔터프라이즈 애플리케이션을 광범위하게 지원합니다.

미디어 렌더링의 경우 **Compute Engine** 인스턴스에 **Filestore** 파일 공유를 쉽게 마운트하여 여러 명의 시각 효과 아티스트가 동일한 파일 공유 항목에서 공동작업할 수 있습니다. 일반적으로 공유 파일 시스템을 마운트하는 컴퓨팅 머신 집합('렌더링 팜') 전반에서 렌더링 워크플로가 실행되므로 **Filestore**와 **Compute Engine**이 작업 렌더링 요구사항에 맞게 확장됩니다.

전자 설계 자동화(**EDA**)는 데이터 관리에 관한 것입니다. 여기에는 수천 개의 코어에서 워크로드를 일괄 처리하는 기능과 대규모 메모리가 필요합니다. **Filestore**는 집약적인 **EDA**를 수행하는 제조사 고객의 요구사항을 충족하는 데 필요한 용량과 규모를 제공하며 어디서든 파일에 액세스할 수 있도록 지원합니다.

데이터 분석 워크로드에는 복잡한 재무 모델 계산 또는 환경 데이터 분석이 포함됩니다. 이러한 워크로드는 지연 시간에 민감합니다. **Filestore**는 파일 작업에 대한 지연 시간이 짧으며 용량 또는 성능 요구사항이 변경되면 필요에 따라 인스턴스를 쉽게 확장하거나 축소할 수 있습니다. 공유 가능한 영구 스토리지 레이어인 **Filestore**는 클라이언트의 드라이브에 데이터를 로드하고 다시 가져오는 데 소중한 시간을 낭비할 필요가 없도록

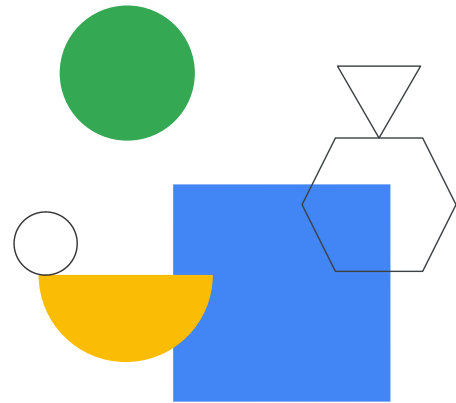
고성능 스마트 분석을 위한 즉각적인 액세스를 제공합니다.

게놈 시퀀싱의 경우, 방대한 원시 데이터가 필요하며 한 사람당 데이터 포인트가 수십억 개에 달합니다. 이러한 유형의 분석에는 속도, 확장성, 보안이 필요합니다. **Filestore**는 성능에 따라 예측 가능한 가격으로 과학 연구를 수행하는 회사 및 연구 기관의 요구사항을 충족합니다.

여러 웹 개발자와 대규모 호스팅 업체가 **Filestore**를 사용하여 **WordPress** 호스팅 등의 요구사항도 충족하면서 웹 콘텐츠를 관리하고 제공합니다.

실습 소개

Cloud Storage



Google Cloud

방금 논의한 Cloud Storage 개념 중 일부를 실습에서 적용해 보겠습니다.

실습 목표

- 01 버킷 생성 및 사용
- 02 액세스 제어 목록(ACL)을 설정하여 액세스 제한
- 03 자체 암호화 키 사용
- 04 버전 제어 구현
- 05 디렉터리 동기화 사용
- 06 IAM을 사용하여 여러 프로젝트에서 버킷 공유



Google Cloud

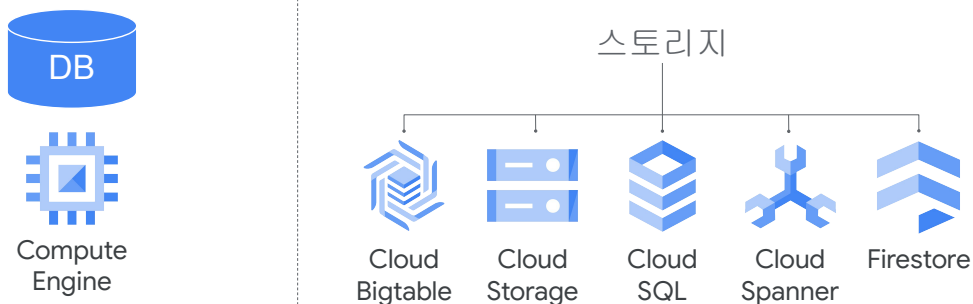
이 실습에서는 버킷을 만들고 **Cloud Storage**에서 사용할 수 있는 여러 가지 고급 옵션을 수행합니다. 그리고 액세스 제어 목록을 설정하여 데이터에 액세스할 수 있는 사람과 데이터로 수행할 수 있는 작업을 제한합니다. 또한 추가적인 보안을 위해 자체 암호화 키를 제공하고 관리하는 기능을 사용해 봅니다. 그런 다음, 객체 버전 관리를 사용하여 데이터의 변경사항을 추적하고 객체가 지정된 기간 후에 자동으로 보관처리되거나 삭제되도록 수명 주기 관리를 구성합니다. 마지막으로 앞서 언급한 디렉터리 동기화 기능을 사용해 보고, **IAM**을 사용하여 프로젝트 간에 버킷을 공유합니다.



Cloud SQL

구조화된 데이터베이스 서비스 또는 관계형 데이터베이스 서비스에 대해 알아보겠습니다.
먼저 Cloud SQL입니다.

자체 데이터베이스 솔루션 구축 또는 관리형 서비스 사용



Google Cloud

Compute Engine을 사용하여 VM에 SQL Server 애플리케이션 이미지를 설치할 수 있는데 왜 SQL용 Google Cloud 서비스를 사용할까요?

사실 문제는 자체 데이터베이스 솔루션을 구축해야 하느냐, 아니면 관리형 서비스를 사용해야 하느냐입니다. 관리형 서비스를 사용하면 얻게 되는 이점이 있습니다. 그러면 Cloud SQL을 Google Cloud 내에서 관리형 서비스로 사용하는 이유에 대해 알아보까요?

완전 관리형 데이터베이스 서비스인 Cloud SQL (MySQL, PostgreSQL, 또는 Microsoft SQL Server)



Cloud SQL

- 패치 및 업데이트가 자동으로 적용됩니다.
- MySQL 사용자를 관리합니다.
- Cloud SQL은 다양한 클라이언트를 지원합니다.
 - `gcloud sql`
 - App Engine, Google Workspace 스크립트
 - 애플리케이션 및 도구
 - SQL Workbench, Toad
 - 표준 MySQL 드라이버를 사용하는 외부 애플리케이션

Google Cloud

Cloud SQL은 MySQL, PostgreSQL, Microsoft SQL Server 데이터베이스의 완전 관리형 서비스입니다. 패치와 업데이트가 자동으로 적용되지만 이러한 데이터베이스와 함께 제공되는 기본 인증 도구를 사용하여 MySQL 사용자를 여전히 관리해야 합니다.

Cloud SQL은 Cloud Shell, App Engine, Google Workspace 스크립트와 같은 다양한 클라이언트를 지원합니다. 또한 표준 MySQL 드라이버를 사용하는 SQL Workbench, Toad 및 기타 외부 애플리케이션과 같이 익숙한 다른 애플리케이션과 도구도 지원합니다.

Cloud SQL 인스턴스

성능:

- 저장용량 64TB
- 60,000IOPS
- 624GB RAM
- 읽기 복제본으로 수평 확장

선택:

- MySQL 5.6, 5.7 또는 8.0(기본)
- PostgreSQL 9.6, 10, 11, 12, 13, 14 또는 15(기본)
- Microsoft SQL Server 2017 또는 2019(표준 기본)



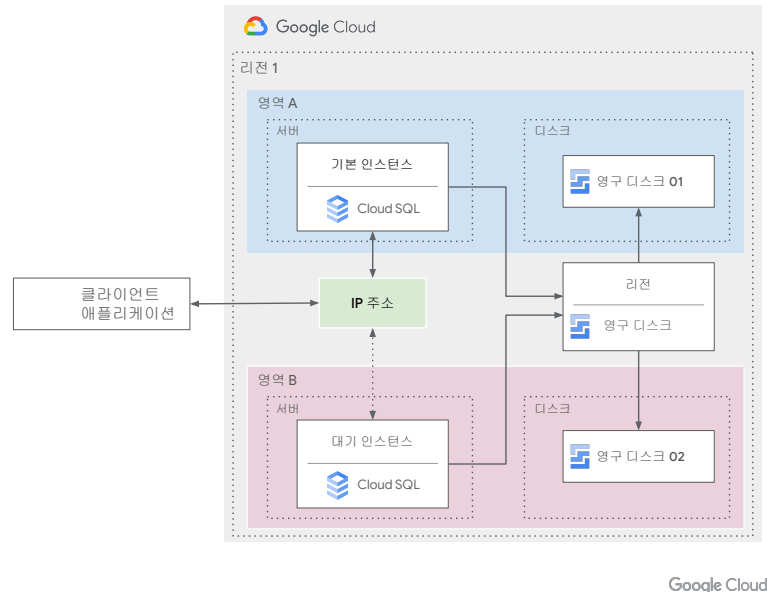
Google Cloud

Cloud SQL은 최대 64TB의 스토리지 용량, 60,000IOPS, 인스턴스당 624GB RAM으로 우수한 성능과 확장성을 제공합니다. 최대 96개의 프로세서 코어까지 쉽게 수직 확장하고 읽기 복제본으로 수평 확장할 수 있습니다.

현재 MySQL 5.6, 5.7, 8.0, PostgreSQL 9.6, 10, 11, 12, 13, 14, 15 또는 Web, Express, Standard, Enterprise SQL Server 2017 또는 2019 버전에서 Cloud SQL을 사용할 수 있습니다.

Cloud SQL 서비스

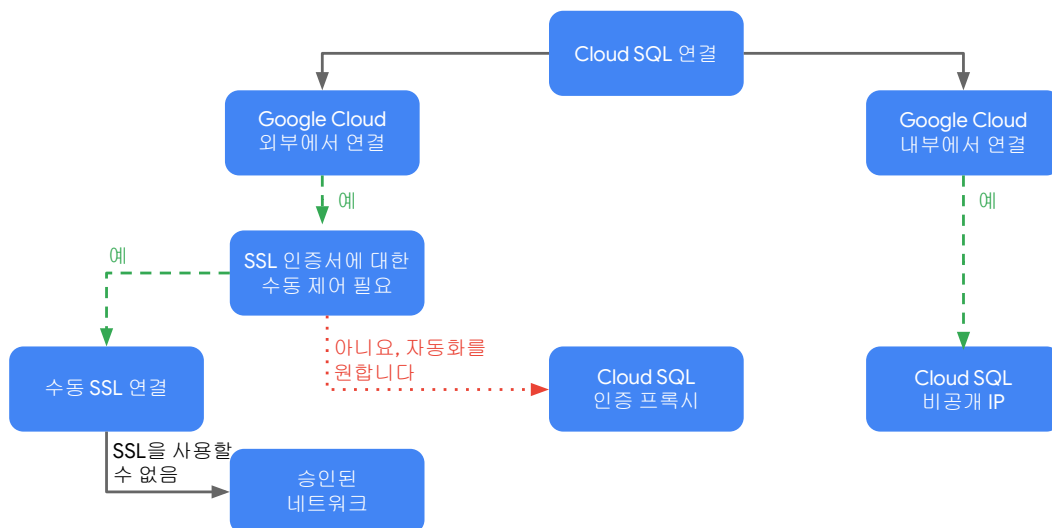
- HA 구성
- 백업 서비스
- 가져오기/내보내기
- 확장
 - 수직: 머신 용량
 - 수평: 읽기 복제본



Cloud SQL에서 제공하는 다른 서비스에 초점을 맞춰 보겠습니다.

- HA 구성에서 리전 인스턴스 내에서의 구성은 기본 인스턴스와 대기 인스턴스로 이루어집니다. 각 영역의 영구 디스크에 동기식으로 복제하는 기능을 통해, 기본 인스턴스에 대한 모든 쓰기 작업은 트랜잭션이 커밋된 것으로 보고되기 전에 양쪽 영역의 디스크에 모두 복제됩니다. 인스턴스 또는 영역에 장애가 발생하면 영구 디스크가 대기 인스턴스에 연결되고 이것이 새 기본 인스턴스가 됩니다. 그런 다음 사용자는 새 기본 인스턴스로 다시 라우팅됩니다. 이 절차를 장애 조치라고 합니다.
- Cloud SQL은 또한 PITR(point-in-time recovery)을 통해 자동화된 주문형 백업을 제공합니다.
- mysqldump를 사용하여 데이터베이스를 가져오고 내보내거나 CSV 파일을 가져오고 내보낼 수 있습니다.
- Cloud SQL도 수직 확장할 수 있으며, 이를 위해서는 머신을 다시 시작하거나 읽기 복제본을 사용하여 수평 확장해야 합니다. 즉, 수평적 확장성에 대해 우려하는 경우 이 모듈의 뒷부분에서 다룰 Cloud Spanner를 고려하는 것이 좋습니다.

Cloud SQL 인스턴스에 연결



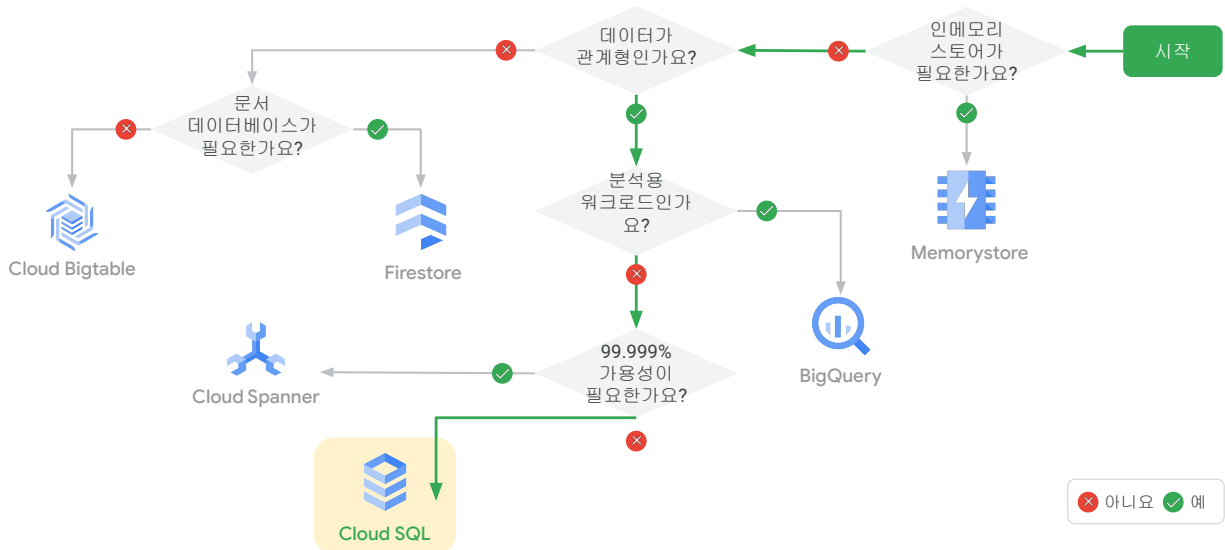
Google Cloud

Cloud SQL 인스턴스에 대한 연결을 어떤 유형으로 선택하느냐에 따라 인스턴스의 보안, 성능, 자동화 수준이 영향을 받게 됩니다. Cloud SQL 인스턴스와 동일한 Google Cloud 프로젝트 내에서 호스팅되는 애플리케이션을 연결하고 동일한 리전에 배치한 경우, 비공개 IP 연결을 선택하면 비공개 연결을 사용하는 가장 효과적이며 안전한 연결이 제공됩니다. 즉, 트래픽이 공개 인터넷에 노출되지 않습니다. 참고: 동일한 리전의 VM에서 Cloud SQL 비공개 IP 주소에 연결하는 것은 성능에 기초한 권장사항일 뿐이지 요구사항이 아닙니다.

애플리케이션이 다른 리전이나 프로젝트에서 호스팅되거나 Google Cloud 외부에서 Cloud SQL 인스턴스에 연결하려는 경우 세 가지 옵션이 있습니다. 이 경우 인증, 암호화 및 키 순환을 처리하는 Cloud SQL 인증 프록시를 사용하는 것이 좋습니다. SSL 연결을 수동으로 제어해야 한다면 인증서를 직접 생성하고 주기적으로 순환할 수 있습니다. 그렇지 않으면 외부 IP 주소를 통해 SQL 서버에 연결할 특정 IP 주소를 승인하여 암호화되지 않은 연결을 사용할 수 있습니다.

다음 실습에서 이러한 옵션을 살펴보겠지만 비공개 IP에 대해 자세히 알아보려면 [문서](#)를 참고하세요.

Cloud SQL 선택하기



Google Cloud

완전한 관계형 기능을 갖춘 올바른 데이터 스토리지 서비스를 찾는 데 도움이 되는 이 결정 트리를 살펴보면서 요약하겠습니다.

Memorystore는 게임 환경 및 실시간 웹로그 분석과 같이 마이크로초 단위의 응답 시간이 필요하거나 트래픽 급증이 발생하는 워크로드를 위한 완전 관리형 인메모리 데이터 스토어 서비스를 제공합니다.

인메모리 데이터 스토어가 필요하지 않지만 사용 사례가 주로 분석에 사용되는 관계형 데이터인 경우 이러한 워크로드는 **BigQuery**에서 가장 효과적으로 지원됩니다.

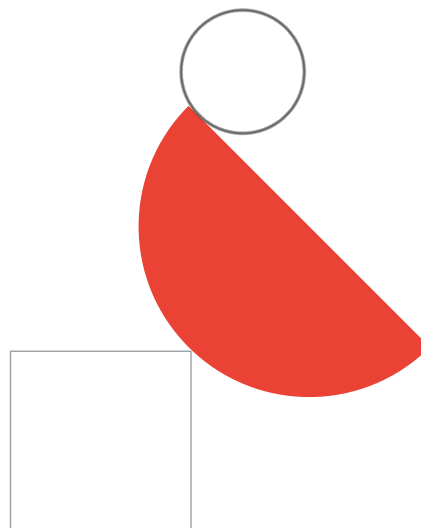
그러나 관계형 데이터 워크로드가 분석이 아니고 **ACID** 속성이 포함된 데이터베이스가 요구사항인 경우 **Cloud Spanner**와 **Cloud SQL** 중에서 선택할 수 있습니다.

수평 확장이나 전 세계적으로 사용 가능한 시스템이 요구되지 않는 경우에는 **Cloud SQL**이 비용 효율이 높은 솔루션입니다.

Cloud SQL을 관리형 서비스로 사용하는 것이 기존 **MySQL** 솔루션을 사용하거나 다시 구현하는 것보다 나은 경우 [MySQL에서 Cloud SQL로 마이그레이션하는 방법을 설명하는 이 솔루션](#)을 참고하세요.

실습 소개

Cloud SQL 구현



Google Cloud

방금 논의한 Cloud SQL 개념 중 일부를 실습에서 적용해 보겠습니다.

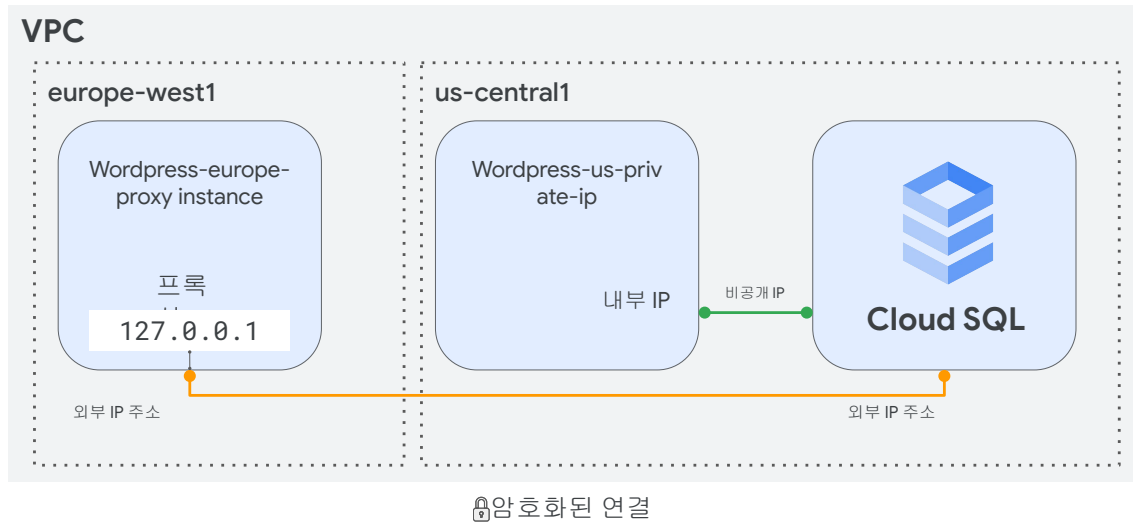
실습 목표

- 01 Cloud SQL 데이터베이스 만들기
- 02 프록시를 실행하도록 가상 머신 구성
- 03 애플리케이션과 Cloud SQL 사이에 연결 만들기
- 04 비공개 IP 주소를 사용하여 Cloud SQL에 애플리케이션 연결



Google Cloud

이 실습에서는 **Cloud SQL** 서버를 구성하고 외부 연결을 사용하여 프록시를 통해 애플리케이션에 연결하는 방법을 알아봅니다. 또한 성능과 보안상의 이점이 뛰어난 비공개 IP 링크를 통해 연결을 구성합니다. 이 실습에서 데모에 사용할 앱은 **Wordpress**이지만 이 실습의 정보와 권장사항은 **SQL** 서버가 필요한 어느 애플리케이션에나 적용할 수 있습니다.



이 실습을 마치면 이 다이어그램에 나와 있는 것과 같이 서로 다른 연결 유형 2개로 SQL 인스턴스 백엔드에 연결된 Wordpress 프론트엔드의 작업 인스턴스 2개가 생성됩니다.



Cloud Spanner

수평적 확장성이 필요하여 Cloud SQL이 요구사항에 맞지 않는 경우 Cloud Spanner 사용을 고려하세요.

관계형 데이터베이스 구조의 이점과 비관계형 수평적 확장을 결합하는 **Cloud Spanner**



Cloud Spanner

- 페타바이트로 확장
- strong consistency
- 고가용성
- 재무 및 인벤토리 애플리케이션에 사용
- 월간 업타임
 - 멀티 리전: 99.999%
 - 리전: 99.99%

Google Cloud

Cloud Spanner는 관계형 데이터베이스 구조와 비관계형 수평적 확장의 이점을 결합하기 위해 클라우드용으로 특별히 구축된 서비스입니다.

이 서비스는 페타바이트 규모의 용량을 제공할 수 있으며 글로벌 규모의 트랜잭션 일관성, 스키마, **SQL** 및 고가용성을 위한 자동 동기식 복제를 제공합니다. 사용 사례에는 전통적으로 관계형 데이터베이스 기술을 사용했던 재무 애플리케이션 및 인벤토리 애플리케이션이 있습니다.

멀티 리전 인스턴스를 생성하는지 아니면 리전 인스턴스를 생성하는지에 따라 이 슬라이드에 나와 있는 것처럼 월간 업타임 **SLA**가 달라집니다. 그러나 최신 수치를 확인하려면 항상 [문서](#)를 참고해야 합니다.

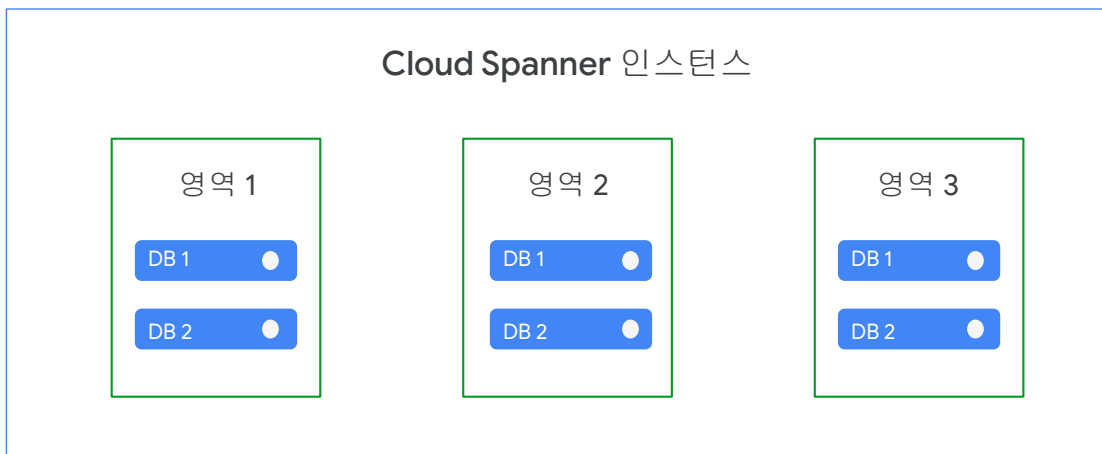
특성

	Cloud Spanner		관계형 DB		비관계형 DB	
스키마	✓	예	✓	예	✗	아니요
SQL	✓	예	✓	예	✗	아니요
일관성	✓	Strong	✓	Strong	✗	Eventual
가용성	✓	높음	✗	장애 조치	✓	높음
확장성	✓	수평	✗	카테고리	✓	수평
복제	✓	자동	↻	구성 가능	↻	구성 가능

Cloud Spanner를 관계형 및 비관계형 데이터베이스와 비교해 보겠습니다. 관계형 데이터베이스와 마찬가지로 Cloud Spanner에는 스키마, SQL, strong consistency가 있습니다. 또한 비관계형 데이터베이스와 마찬가지로 Cloud Spanner는고가용성, 수평적 확장성, 구성 가능한 복제를 제공합니다.

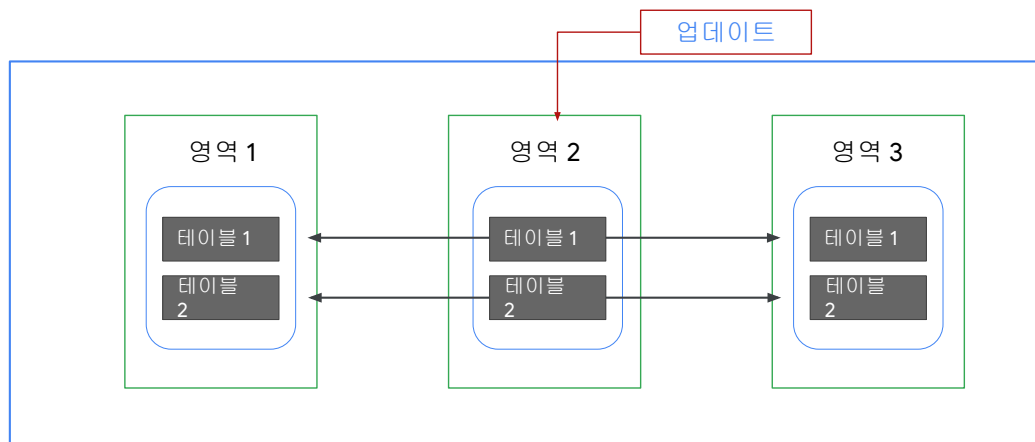
언급했듯이 Cloud Spanner는 최고의 관계형 및 비관계형 환경을 제공합니다. 중요한 사용 사례에 이러한 기능을 사용할 수 있습니다. 금융 서비스 및 소매 산업에서 거래 및 재고 관리를 위한 일관된 시스템을 구축하는 경우를 예로 들 수 있죠. 이 모든 것이 어떻게 작동하는지 더 잘 파악하려면 Cloud Spanner의 아키텍처를 살펴보면 됩니다.

Cloud Spanner 아키텍처



Cloud Spanner 인스턴스는 한 리전 안에 있거나 여러 리전에 걸쳐 있을 수 있는 N개의 클라우드 영역에서 데이터를 복제합니다. 데이터베이스 배치는 구성 가능합니다. 즉, 데이터베이스를 배치할 리전을 선택할 수 있습니다. 이 아키텍처는고가용성과 전역 배치를 허용합니다.

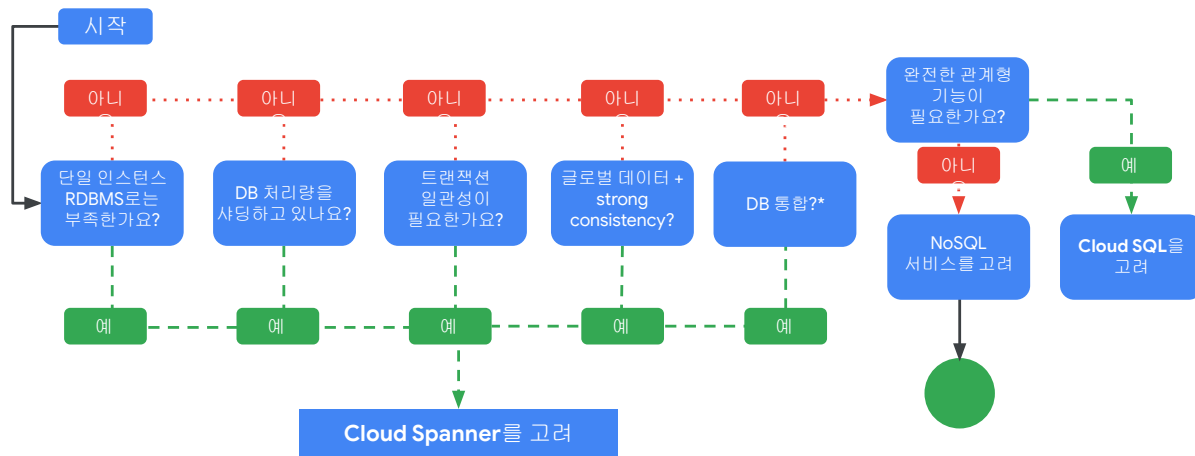
Google의 글로벌 광섬유 네트워크를 사용하여 영역 간에 동기화되는 데이터 복제



데이터 복제는 Google의 글로벌 광섬유 네트워크를 사용하여 영역 간에 동기화됩니다. 원자 시계를 사용하면 데이터를 업데이트할 때마다 전역 일관성이 보장됩니다.

Cloud Spanner에 대한 설명은 여기까지입니다. 이 모듈의 초점은 Cloud Spanner를 사용하는 상황을 이해하는 것이므로 결정 트리를 통해 살펴보겠습니다.

Cloud Spanner 선택하기



관계형 데이터베이스가 부족하거나 고성능 처리량을 위해 데이터베이스를 샅당하거나 트랜잭션 일관성, 글로벌 데이터 및 **strong consistency**가 필요하거나 단순히 데이터베이스를 통합하려는 경우 **Cloud Spanner**를 사용하는 것이 좋습니다.

이들 중 어느 것도 필요하지 않거나 완전한 관계형 기능이 필요하지 않다면 다음에 다른 **Firestore**와 같은 **NoSQL** 서비스를 고려하세요.

Cloud Spanner를 관리형 서비스로 사용하는 것이 기존 **MySQL** 솔루션을 사용하거나 다시 구현하는 것보다 낫다고 확신하는 경우 [MySQL에서 Cloud Spanner로 마이그레이션하는 방법을 설명하는 이 솔루션](#)을 참고하세요.



AlloyDB

이제 AlloyDB에 대해 살펴볼까요?

AlloyDB는 완전 관리형 데이터베이스 서비스입니다



AlloyDB

- 완전 관리형 데이터베이스 서비스
- 빠른 트랜잭션 처리
- 고가용성
- 실시간 비즈니스 인사이트

Google Cloud

PostgreSQL용 AlloyDB는 하이브리드 트랜잭션 및 분석 처리를 비롯한 가장 까다로운 워크로드를 위해 설계된 완전 관리형 PostgreSQL 호환 데이터베이스 서비스입니다. AlloyDB는 Google이 구축한 데이터베이스 엔진과 클라우드 기반의 멀티노드 아키텍처를 결합하여 엔터프라이즈급 성능, 안정성, 가용성을 제공합니다.

AlloyDB는 백업, 복제, 패치, 용량 관리와 같은 관리 작업을 자동화합니다. 또한 AlloyDB는 적응형 알고리즘과 머신러닝을 사용하여 PostgreSQL 데이터 제거 관리, 스토리지 및 메모리 관리, 데이터 계층화, 분석 가속화를 지원합니다.

AlloyDB는 트랜잭션 워크로드에 대해 표준 PostgreSQL보다 4배 더 빠릅니다. 높은 트랜잭션 처리량, 대규모 데이터 크기, 여러 읽기 복제본이 필요한 워크로드를 포함하여 가장 까다로운 엔터프라이즈 워크로드에 적합합니다.

AlloyDB는 유지보수를 포함하여 고가용성과 99.99% 업타임 SLA를 제공합니다. 또한 AlloyDB는 실시간 비즈니스 통계를 제공하며 분석 쿼리에 대해 표준 PostgreSQL보다 최대 100배 빠른 속도를 제공합니다. Google의 인공지능 플랫폼인 Vertex AI와의 기본 통합을 통해 머신러닝 모델을 호출할 수 있습니다.



Firestore

애플리케이션을 위해 확장성 뛰어난 NoSQL 데이터베이스를 찾고 있다면 **Firestore**를 사용해 보세요.

NoSQL 문서 데이터베이스인 **Firestore**



Firestore

- 데이터 저장, 동기화 및 쿼리 간소화
- 글로벌 규모의 모바일, 웹 및 IoT 앱
- 실시간 동기화 및 오프라인 지원
- 보안 기능
- ACID 트랜잭션
- 멀티 리전 복제
- 강력한 쿼리 엔진

Google Cloud

Firestore는 글로벌 규모의 모바일, 웹 및 IoT 애플리케이션용 데이터의 저장, 동기화, 쿼리를 간소화하는 클라우드 네이티브의 완전 관리형 서버리스 고속 **NoSQL** 문서 데이터베이스입니다. 클라이언트 라이브러리는 실시간 동기화와 오프라인 지원을 제공하며, 보안 기능과 **Firebase** 및 **Google Cloud**와의 통합은 진정한 서버리스 앱 빌드를 가속화합니다.

Firestore는 또한 **ACID** 트랜잭션을 지원하므로 트랜잭션의 작업이 하나라도 실패하고 다시 시도할 수 없는 경우 전체 트랜잭션이 실패합니다.

자동 멀티 리전 복제 및 **strong consistency**를 통해 재해가 발생해도 데이터를 안전하게 보호하고 문제없이 사용할 수 있습니다. **Firestore**를 사용하면 성능 저하 없이 **NoSQL** 데이터에 대해 정교한 쿼리를 실행할 수도 있습니다. 따라서 데이터를 보다 유연하게 구성할 수 있습니다.

차세대 Datastore 서비스인 Firestore

Datastore 모드(새 서버 프로젝트):

- Datastore 애플리케이션과 호환
- strong consistency
- 항목 그룹 제한 없음

기본 모드(새 모바일 및 웹 앱):

- strong consistency를 갖춘 스토리지 레이어
- 컬렉션 및 문서 데이터 모델
- 실시간 업데이트
- 모바일 및 웹 클라이언트 라이브러리

Google Cloud

Firestore는 차세대 Datastore 서비스입니다. Firestore는 Datastore 모드로 작동할 수 있으며, Datastore와 하위 호환됩니다. Datastore 모드에서 Firestore 데이터베이스를 만들면 Datastore 시스템 동작을 유지하면서 Firestore의 향상된 스토리지 레이어에 액세스할 수 있습니다.

이렇게 하면 다음과 같은 Datastore 제한사항이 사라집니다.

- 쿼리가 더 이상 eventual consistency를 가지지 않고, strong consistency를 가집니다.
- 트랜잭션이 더 이상 25개의 항목 그룹으로 제한되지 않습니다.
- 항목 그룹에 대한 쓰기가 더 이상 초당 1회로 제한되지 않습니다.

기본 모드의 Firestore는 다음과 같은 새로운 기능을 제공합니다.

- strong consistency를 갖춘 새로운 스토리지 레이어
- 컬렉션 및 문서 데이터 모델
- 실시간 업데이트
- 모바일 및 웹 클라이언트 라이브러리

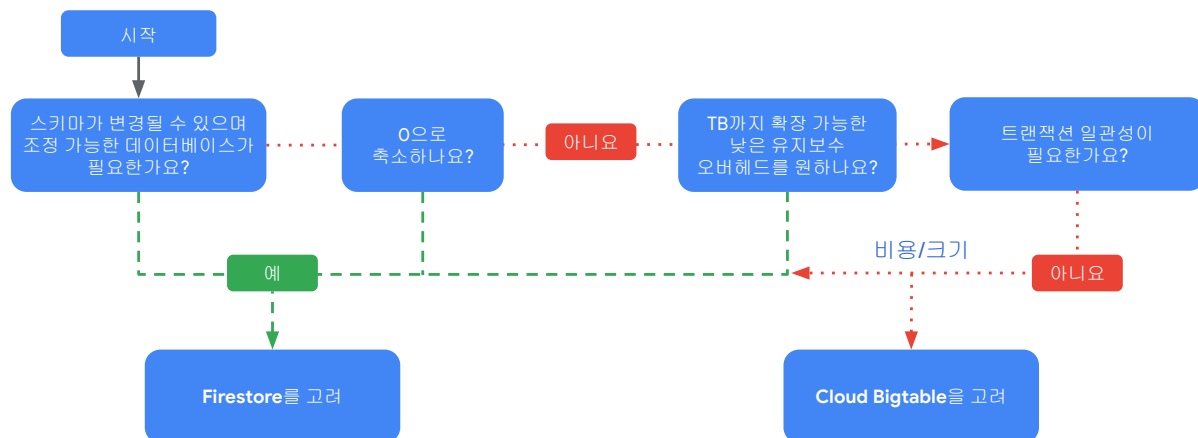
Firestore는 Datastore의 이전 버전과 호환되지만 새로운 데이터 모델, 실시간 업데이트, 모바일 및 웹 클라이언트 라이브러리 기능은 호환되지 않습니다. 새로운 모든 Firestore 기능에 액세스하려면 기본 모드의 Firestore를 사용해야 합니다. 일반적인 지침은 새 서버 프로젝트의 경우 Datastore 모드의 Firestore를 사용하고 새 모바일 및 웹 앱의 경우 기본 모드를 사용하는 것입니다.

차세대 Datastore인 Firestore는 모든 Datastore API 및 클라이언트 라이브러리와 호환됩니다. 기존 Datastore 사용자는 향후 Firestore로 실시간으로 자동 업그레이드됩니다. 자세한 내용은 다음 링크를 참고하세요.

<https://cloud.google.com/datastore/docs/firestore-or-datastore>

<https://cloud.google.com/datastore/docs/upgrade-to-firestore>

Firestore 선택하기



Firestore가 데이터에 적합한 스토리지 서비스인지 판단하는 데 도움이 되는 이 결정 트리를 살펴보면서 요약하겠습니다.

스키마가 변경될 수 있고 조정 가능한 데이터베이스가 필요하거나, 0으로 축소해야 하거나, 최대 테라바이트까지 확장 가능한 낮은 유지보수 오버헤드를 원하는 경우 Firestore를 사용하는 것이 좋습니다.

또한 트랜잭션 일관성이 필요하지 않다면 비용이나 크기에 따라 Cloud Bigtable을 고려하는 것이 좋습니다.

이제 Cloud Bigtable에 대해서 살펴보겠습니다.



Cloud Bigtable

트랜잭션 일관성이 필요하지 않다면 Cloud Bigtable을 고려해볼 수 있습니다.

NoSQL 빅데이터 데이터베이스 서비스인 Cloud Bigtable



Cloud Bigtable

- 페타바이트급 규모
- 10밀리초 미만의 일관된 지연 시간
- 처리량을 위한 완벽한 확장성
- 액세스 패턴 학습 및 조정
- 애드테크, 핀테크, IoT에 이상적
- ML 애플리케이션을 위한 스토리지 엔진
- 오픈소스 빅데이터 도구와 간편하게 통합



Google Cloud

Cloud Bigtable은 매우 짧은 지연 시간을 제공하는 페타바이트 규모의 완전 관리형 NoSQL 데이터베이스입니다. 처리량에 맞게 원활하게 확장되며 특정 액세스 패턴에 적응하는 방법을 학습합니다. Cloud Bigtable은 실제로 검색, 분석, 지도, Gmail을 비롯한 Google의 수많은 핵심 서비스를 운영하는 데이터베이스입니다.

Cloud Bigtable은 IoT, 사용자 분석, 재무 데이터 분석을 비롯한 운영 및 분석 애플리케이션에 모두 적합한데, 그 이유는 지연 시간이 짧으면서도 높은 읽기 및 쓰기 처리량을 지원하기 때문입니다. 또한 머신러닝 애플리케이션을 위한 훌륭한 스토리지 엔진이기도 합니다.

Cloud Bigtable은 Hadoop, Dataflow, Dataproc 등 널리 사용되는 빅데이터 도구와 쉽게 연동됩니다. 또한 Cloud Bigtable은 업계 표준의 오픈소스 HBase API를 지원하기 때문에 개발팀이 쉽게 익힐 수 있습니다. Dataflow와 Dataproc은 이 과정의 후반부에 다룹니다. HBase API에 대한 자세한 내용은 이 [링크](#)를 참고하세요.

Cloud Bigtable 스토리지 모델

'follows' column family

	Follows			
Row Key	gWashington	Jadams	tJefferson	Wmckinley
gWashington		1		
Jadams	1		1	
tJefferson	1	1		1
Wmckinley			1	

다양한 버전

Google Cloud

Cloud Bigtable은 각각 정렬된 키/값 매핑으로 구성되어 있고 대규모로 확장 가능한 테이블에 데이터를 저장합니다. 이 테이블은 일반적으로 단일 항목을 기술하는 행과 각 행의 개별 값을 포함하는 열로 구성됩니다. 각 행은 단일 **row key**로 색인이 생성되고, 서로 연관된 열은 일반적으로 **column family**로 그룹화됩니다. 각 열은 **column family**와 **column family** 내 고유 이름인 **column qualifier**의 조합으로 식별됩니다.

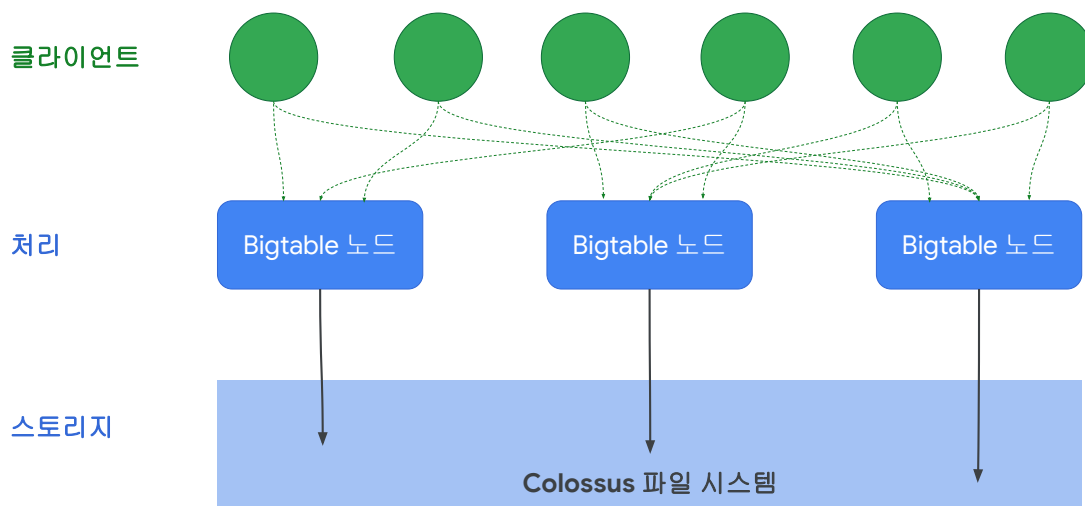
각 행/열 교집합에는 서로 다른 타임스탬프에 다양한 셀 또는 버전이 포함될 수 있으므로 시간에 경과함에 따라 저장된 데이터가 어떻게 바뀌었는지에 대한 기록을 확인할 수 있습니다. Cloud Bigtable 테이블은 희소 테이블이므로, 셀에 데이터가 없으면 공간을 차지하지 않습니다.

여기에 나와 있는 예는 미국 대통령을 위한 가상의 사회 연결망을 가정한 것으로, 이 사회 연결망에서는 각 대통령이 다른 대통령의 게시물을 팔로우할 수 있습니다. 몇 가지를 설명하겠습니다.

- 이 테이블에는 **column family**가 한 개(**follows** 그룹) 있습니다. 이 그룹에는 여러 개의 **column qualifier**가 포함됩니다.
- **column qualifier**는 데이터처럼 사용됩니다. 이러한 디자인 옵션은 Cloud Bigtable 테이블의 희소성과 새로운 **column qualifier**를 데이터가 변경될 때 추가할 수 있는 기능 덕분에 가능합니다.
- 사용자 이름은 **row key**로 사용됩니다. 사용자 이름에 알파벳이 고르게 사용되었다면 전체 테이블에서 데이터 액세스가 고르게 이루어지는 것이

- 당연합니다.

스토리지와 분리된 처리



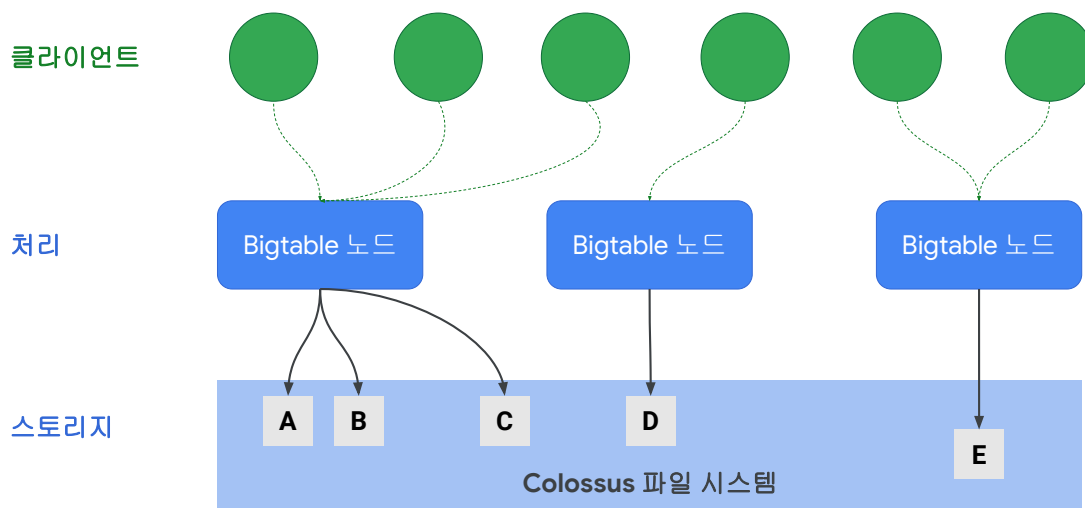
Google Cloud

이 다이어그램은 **Cloud Bigtable**의 전체 아키텍처를 단순화한 버전을 보여 줍니다. 프론트엔드 서버 풀과 노드를 통해 수행되는 처리가 스토리지와는 별도로 처리됨을 보여 줍니다.

Cloud Bigtable 테이블은 태블릿이라고 하는 연속된 행의 블록으로 분할되어 쿼리 워크로드를 분산시킵니다. **HBase API**를 사용한 적이 있는 사용자라면 태블릿은 **HBase** 리전과 유사하다고 생각하시면 됩니다.

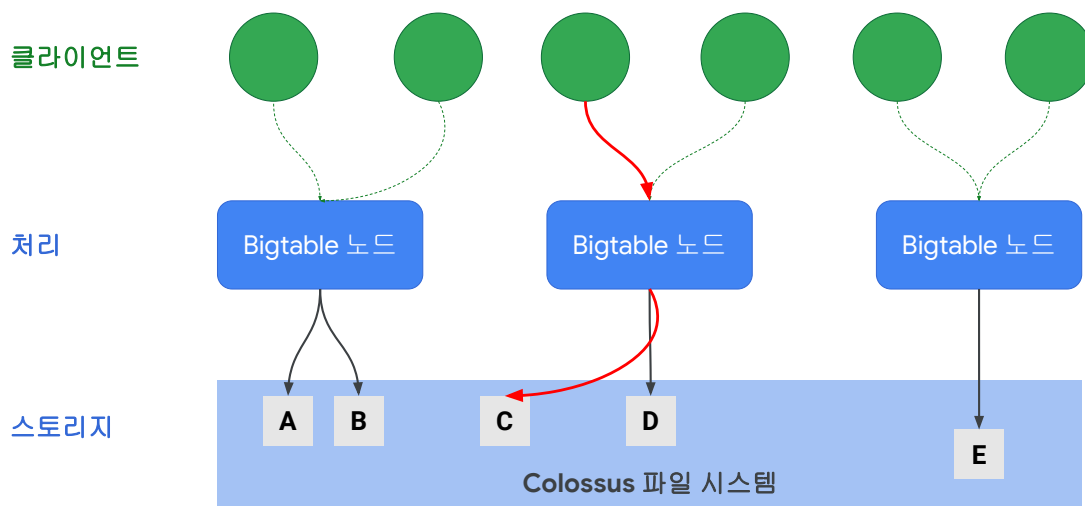
태블릿은 **Google** 파일 시스템인 **Colossus**에 **SSTable** 형식으로 저장됩니다. **SSTable**은 영구적이고 불변하며 정렬된 키-값 매핑을 제공하는데, 키와 값은 모두 임의 바이트 문자열입니다.

액세스 패턴 학습



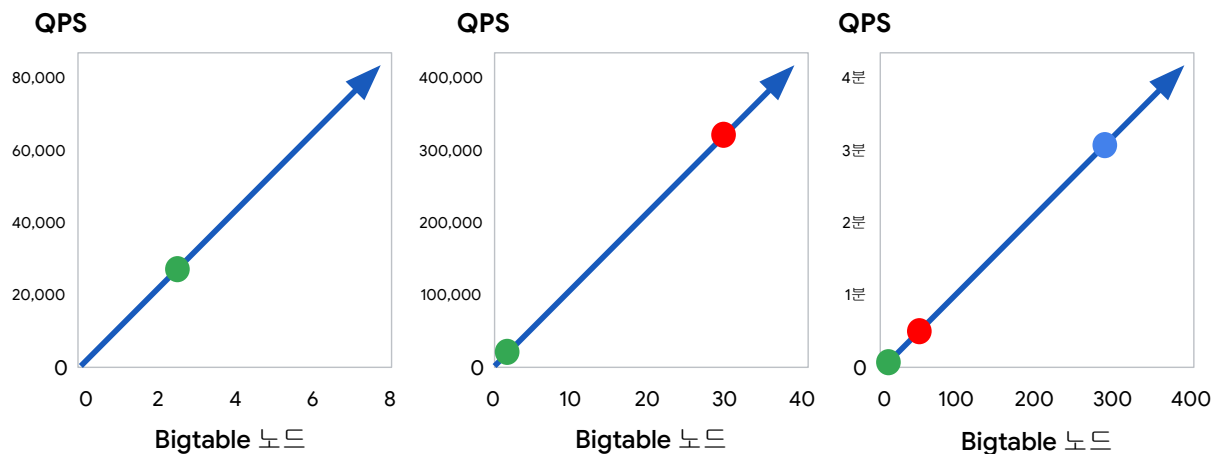
앞서 언급했듯이 Cloud Bigtable은 특정 액세스 패턴에 적응하는 방법을 학습합니다. 특정 Bigtable 노드가 특정 데이터 하위 집합에 자주 액세스하는 경우...

데이터 이동 없이 재조정



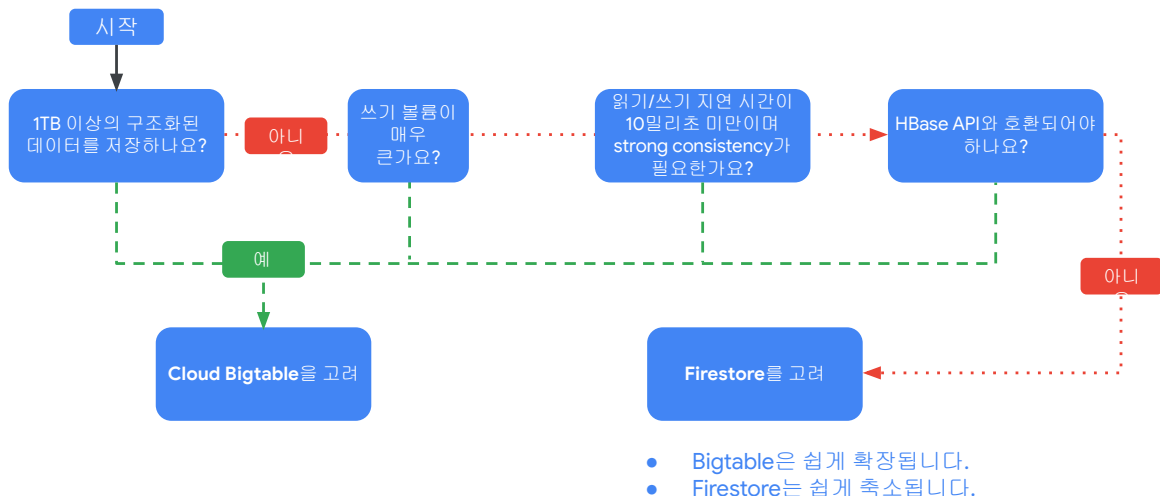
Cloud Bigtable은 여기서 볼 수 있는 것처럼 다른 노드가 해당 워크로드를 고르게 분배할 수 있도록 색인을 업데이트합니다.

선형으로 확장되는 처리량



처리량은 선형으로 확장됩니다. 따라서 추가하는 모든 단일 노드에 대해 최대 수백 개의 노드까지 처리량 성능이 선형으로 확장되는 것을 볼 수 있습니다.

Cloud Bigtable 선택하기



요약하자면, 1TB 이상의 구조화된 데이터를 저장해야 하거나, 쓰기 볼륨이 매우 크거나, strong consistency와 함께 10밀리초 미만의 읽기/쓰기 지연 시간이 필요하거나, HBase API와 호환되는 스토리지 서비스가 필요한 경우 Cloud Bigtable을 사용하는 것이 좋습니다.

이들 중 어느 것도 필요하지 않고 쉽게 축소되는 스토리지 서비스를 찾고 있다면 Firestore를 사용하는 것이 좋습니다.

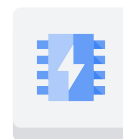
확장에 대해 말하자면, 생성할 수 있는 가장 작은 Cloud Bigtable 클러스터에는 3개의 노드가 있고 초당 30,000개의 작업을 처리할 수 있습니다. 애플리케이션에서 노드를 사용하는지 여부와 관계없이 노드가 작동하는 동안 해당 노드에 대해 비용을 지불하게 됩니다.



Memorystore

Memorystore에 대해 간단하게 설명하겠습니다.

완전 관리형 Redis 서비스인 Memorystore



Memorystore

- 인메모리 데이터 스토어 서비스
- 뛰어난 앱 빌드에 집중 가능
- 고가용성, 장애 조치, 패치 및 모니터링
- 밀리초 미만의 지연 시간
- 최대 300GB의 인스턴스
- 12Gbps의 네트워크 처리량
- 간단한 리프트 앤 시프트

Google Cloud

Redis용 Memorystore는 Google에서 관리하는 확장 가능하고 안전하며 가용성이 높은 인프라를 기반으로 한 완전 관리형 인메모리 데이터 스토어 서비스를 제공합니다. Google Cloud에서 실행되는 애플리케이션은 복잡한 Redis 배포를 관리할 필요 없이 확장성, 가용성, 안전성이 뛰어난 Redis 서비스를 활용하여 성능을 극대화할 수 있습니다. 이렇게 하면 코드를 작성하는 데 더 많은 시간을 투자할 수 있으므로 훌륭한 앱을 빌드하는 데 집중할 수 있습니다.

또한 Memorystore는 고가용성, 장애 조치, 패치, 모니터링과 같은 복잡한 작업을 자동화합니다. 고가용성 인스턴스는 2개의 영역에 복제되며 99.9% 가용성의 SLA를 제공합니다.

따라서 애플리케이션에 필요한 밀리초 이하의 지연 시간과 처리량을 쉽게 달성할 수 있습니다. 최저 등급과 최소 크기로 시작한 후 애플리케이션 가용성에 대한 영향을 최소화하면서 간편하게 인스턴스를 확장해 보세요. Memorystore는 최대 300GB의 인스턴스와 12Gbps의 네트워크 처리량을 지원할 수 있습니다.

Redis용 Memorystore는 Redis 프로토콜과 완벽하게 호환되므로 가져오기/내보내기 기능을 사용하여 코드 변경 없이 애플리케이션을 오픈소스 Redis에서 Memorystore로 리프트 앤 시프트할 수 있습니다. 모든 기존 도구와 클라이언트 라이브러리가 작동하기 때문에 새 도구를 익히지 않아도 됩니다.



퀴즈



질문 1

질문

데이터 센터의 단일 머신에서 실행되는 표준 관계형 데이터베이스를 클라우드로 마이그레이션해야 하는 경우 어떤 데이터 스토리지 서비스를 선택해야 할까요?

- A. Cloud SQL
- B. BigQuery
- C. Persistent Disk
- D. Cloud Storage

질문 1

답변

데이터 센터의 단일 머신에서 실행되는 표준 관계형 데이터베이스를 클라우드로 마이그레이션해야 하는 경우 어떤 데이터 스토리지 서비스를 선택해야 할까요?

- A. Cloud SQL
- B. BigQuery
- C. Persistent Disk
- D. Cloud Storage



설명:

Cloud SQL은 PostgreSQL 서버 또는 MySQL 서버를 관리형 서비스로 제공합니다.

질문 2

질문

ACID 트랜잭션을 제공하고 글로벌 단위로 확장할 수 있는 Google Cloud 데이터 스토리지 서비스는 무엇인가요?

- A. Cloud Storage
- B. Cloud CDN
- C. Cloud Spanner
- D. Cloud SQL

질문 2

답변

ACID 트랜잭션을 제공하고 글로벌 단위로 확장할 수 있는 Google Cloud 데이터 스토리지 서비스는 무엇인가요?

- A. Cloud Storage
- B. Cloud CDN
- C. Cloud Spanner
- D. Cloud SQL



설명:

Cloud Spanner는 데이터베이스에서 트랜잭션 읽기 및 쓰기를 가능하게 하는 ACID(원자성, 일관성, 격리, 내구성) 속성을 제공합니다. 또한 글로벌 단위로 확장할 수 있습니다.

질문 3

질문

다음 중 데이터 저장을 위한 데이터 웨어하우스 서비스뿐만 아니라 데이터 쿼리를 위한 대화형 SQL 인터페이스까지 제공하는 데이터 스토리지 서비스는 무엇인가요?

- A. BigQuery
- B. Dataproc
- C. Datalab
- D. Cloud SQL

질문 3

답변

다음 중 데이터 저장을 위한 데이터 웨어하우스 서비스뿐만 아니라 데이터 쿼리를 위한 대화형 SQL 인터페이스까지 제공하는 데이터 스토리지 서비스는 무엇인가요?

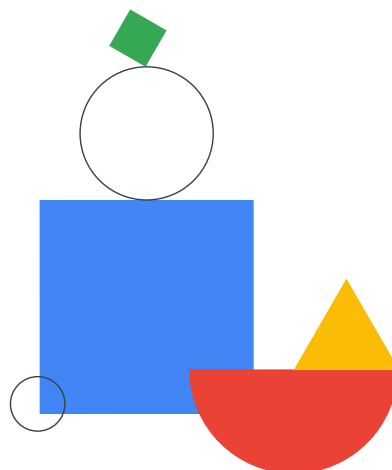
- A. BigQuery
- B. Dataproc
- C. Datalab
- D. Cloud SQL

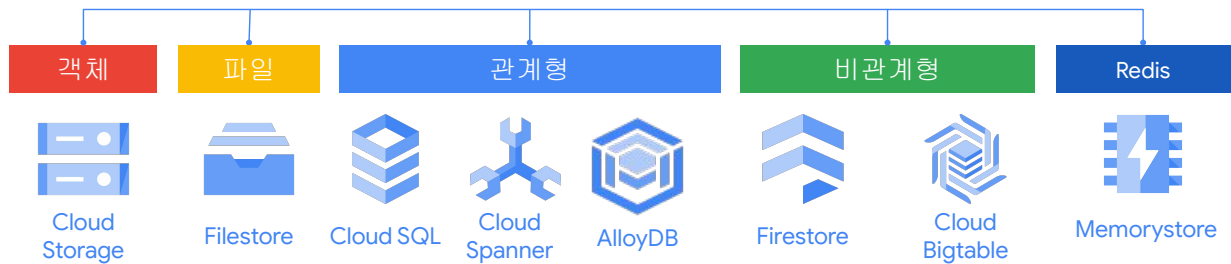


설명:

BigQuery는 대규모 데이터 세트를 저장하면서도, 별도의 서비스에서 처리를 추출하거나 실행할 필요 없이 즉시 처리할 수 있도록 지원하는 데이터 웨어하우스 서비스입니다.

리뷰: 스토리지 및 데이터베이스 서비스

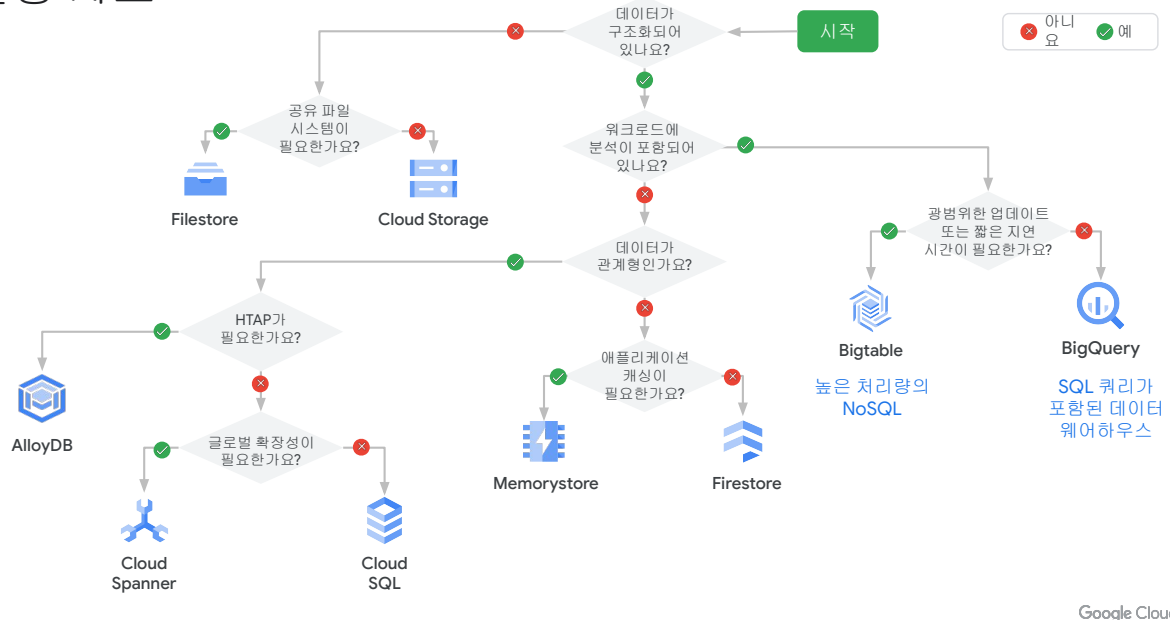




이 모듈에서는 Google Cloud가 제공하는 다양한 스토리지 및 데이터베이스 서비스를 다루었습니다. 특히 완전 관리형 객체 저장소인 Cloud Storage, 완전 관리형 파일 저장 서비스인 Filestore, 완전 관리형 MySQL/PostgreSQL 데이터베이스 서비스인 Cloud SQL, 트랜잭션 일관성, 글로벌 확장성 및 고가용성을 갖춘 관계형 데이터베이스 서비스인 Cloud Spanner, 완전 관리형 PostgreSQL 호환 데이터베이스 서비스인 AlloyDB, 완전 관리형 NoSQL 문서 데이터베이스인 Firestore, 완전 관리형 NoSQL 와이드 칼럼 데이터베이스인 Cloud Bigtable, Redis용 완전 관리형 인메모리 데이터 스토어 서비스인 Memorystore에 대해 배웠습니다.

인프라 관점에서의 목표는 사용 가능한 서비스가 무엇인지, 다양한 상황에서 서비스가 어떻게 사용되는지를 이해하는 것이었습니다. 완전한 데이터 전략을 정의하는 방법은 이 과정에서 다루지 않지만 Google Cloud에서의 데이터 엔지니어링 및 머신러닝 교육 과정에서 데이터 전략에 대해 다룹니다. 자세한 내용은 [Google Cloud 교육 사이트](#)를 참고하세요.

결정 차트



이 결정 차트를 토대로 이 모듈에서 소개한 서비스를 요약해 보겠습니다.

- 먼저 데이터가 구조화되어 있는지 자문합니다. 구조화되어 있지 않다면 공유 파일 시스템이 필요한지 자문해 보세요. 공유 파일 시스템이 필요하다면 **Filestore**를 선택합니다.
- 필요하지 않다면 **Cloud Storage**를 선택합니다.
- 데이터가 구조화되어 있다면 워크로드가 분석에 초점이 맞춰져 있는지 생각해 보세요. 답이 '예'라면 짧은 지연 시간 및 업데이트 필요 여부에 따라 **Bigtable** 또는 **BigQuery**를 선택합니다.
 - **BigQuery**는 데이터 웨어하우스로 권장되고, 테이블 형식 데이터의 기본 스토리지이며, 대규모 임시 **SQL** 기반 분석 및 보고에 최적화되어 있습니다. **BigQuery DML**을 사용하면 **BigQuery** 테이블에서 데이터를 업데이트, 삽입, 삭제할 수 있습니다. **BigQuery DML**에는 기본 제공 캐시가 있어 데이터가 자주 변경되지 않는 경우에도 **BigQuery**는 아주 잘 작동합니다.
 - **Bigtable**은 **NoSQL** 와이드 칼럼 데이터베이스입니다. **Bigtable**은 짧은 지연 시간, 대량의 읽기 및 쓰기, 대규모 성능 유지에 최적화되어 있습니다.
 - 분석 외에도 **Bigtable**은 메모리에 저장하기에는 너무 큰 데이터 세트를 위한 '빠른 조회' 비관계형 데이터베이스로도 적합하며 IoT, 애드테크, 핀테크 등의 분야에서 사용 사례가 있습니다.
- 워크로드에 분석이 포함되지 않은 경우 데이터가 관계형인지 확인하세요. 관계형이 아니지만 애플리케이션 캐싱이 필요한가요?
 - 캐싱이 필요한 경우 인메모리 데이터베이스인 **Memorystore**를 선택하세요.
 - 그렇지 않은 경우 문서 데이터베이스인 **Firestore**를 선택하세요.
- 데이터가 관계형이고 **HTAP**(하이브리드 트랜잭션/분석 처리)가 필요한 경우 **AlloyDB**를 선택하세요.

- HTAP(하이브리드 트랜잭션/분석 처리)와 글로벌 확장성이 필요하지 않은 경우 **Cloud SQL**을 선택하세요.
- HTAP(하이브리드 트랜잭션/분석 처리)가 필요하지 않고 글로벌 확장성이 필요한 경우 **Spanner**를 선택하세요.

애플리케이션에 따라 이러한 서비스 중 하나 또는 여러 개를 사용하여 작업을 완료할 수 있습니다. 이렇게 다양한 서비스 중에서 적절한 서비스를 선택하는 방법에 대한 자세한 내용은 다음 두 링크를 참고하세요.

<https://cloud.google.com/storage-options/>

<https://cloud.google.com/products/databases/>

